L'IA AU SERVICE DE L'ÉCOLOGIE

TRI FACILE

Projet sciences des données 4 - Université de Montpellier Paul Valéry

Ambre Faizandier
Nina Valentin
Anthony Miranda
Victor Croenne

Remerciements

La réalisation de ce projet a été une aventure collective enrichissante qui a mobilisé les compétences et les idées de chacun des membres de notre équipe. Au-delà de notre investissement personnel, ce travail a bénéficié du soutien et de l'expertise de plusieurs personnes à qui nous tenons à exprimer notre profonde gratitude.

Nous tenons tout d'abord à remercier l'équipe pédagogique de l'UE Sciences des données 4 : Mme Bringay, Mme Patel et M. Michel-Picque. Nous les remercions pour leur patience, leur disponibilité et leurs précieux conseils méthodologiques qui nous ont permis de structurer notre réflexion et de mener à bien ce projet complexe.

Nous adressons également nos sincères remerciements à Mme Lèbre. Bien que n'intervenant pas directement dans cette unité d'enseignement, elle a accepté avec générosité de nous accorder de son temps. Ses suggestions sur le choix des modèles et des évaluations des modèles ont été déterminantes dans la fiabilité de nos modèles.

Enfin, nous souhaitons remercier l'Université de Montpellier Paul-Valéry pour les ressources mises à notre disposition, ainsi que toutes les personnes qui, de près ou de loin, ont contribué par leurs échanges et leurs encouragements à la réussite de ce projet.

Résumé

Tri Facile est une plateforme innovante conçue pour optimiser la gestion des déchets en Italie. Dans un contexte où les communes peinent souvent à évaluer l’impact réel de leurs politiques environnementales, notre outil mobilise l’intelligence artificielle pour transformer l’Open Data en un véritable levier de décision.

Le projet s'articule autour d'une approche complémentaire alliant analyse prédictive et accompagnement citoyen. D'une part, un simulateur fondé sur le modèle Random Forest et les régressions linéaires et logistiques, permet aux municipalités d'estimer précisément leurs flux de déchets en fonction de critères socio-économiques et de l'efficacité des redevances incitatives. D'autre part, un assistant intelligent basé sur un réseau de neurones (CNN) facilite le geste de tri quotidien des habitants grâce à la reconnaissance d'images. En combinant ainsi l'analyse statistique et la vision par ordinateur, Tri Facile offre une solution concrète et intégrée pour piloter la transition écologique à l'échelle locale.

Sommaire

1Introduction **9**

1.2 Contexte 9

1.3 Problématique 9

1.4 Objectif 10

22 Matériel et méthodes **11**

2.1 Logiciels 11

2.2 Librairies 12

2.3 Ordinateur utilisés 13

33 Bases de Données **15**

3.1 Provenance des données 15

3.2 Descriptif des données 15

3.3 Modèles MCD et MOD 17

3.4 Nettoyage 18

44 Analyses exploratoire des données **19**

4.1 Visualisation 20

55 Machine Learning **29**

5.1 Base de données 1 : Prédictions communales 29

5.2 Base de données 2 : Reconnaissance d'images (CNN) 33

66 Analyse et Résultats **42**

6.1 Base de données 1 : Machine Learning 42

6.2 Base de données 2 : CNN 48

77 Site Web **56**

7.2 Fonctionnalités 58

88 Conclusion et Perspectives **62**

8.1 Conclusion 62

8.2 Perspectives 62

Chapitre 1

Introduction

1.2 Contexte

La gestion des déchets est devenue l'un des plus gros défis écologiques de notre époque. En Italie, comme c'est le cas dans beaucoup de pays en Europe, on remarque que la situation est très différente d'une ville à l'autre à cause du niveau de vie ou des règles locales qui changent. Aujourd'hui, les mairies ont besoin de trouver des solutions pour mieux trier et surtout pour réduire les coûts. Pour cela, certaines communes mettent en place une “redevance incitative”, c'est-à-dire qu'elles font payer leurs habitants selon le poids ou le volume de leurs poubelles. Seulement pour les élus, il est difficile de prédire si cette décision va vraiment marcher ou non. Enfin, pour les habitants, ce n'est pas toujours simple de savoir dans quelle poubelle jeter chaque déchet. Dans ce contexte, nous avons trouvé intéressant de créer un outil qui utilise l'intelligence artificielle pour aider d’une part, les mairies à prévoir leurs flux de déchets et d’autre part les citoyens à mieux trier.

1.3 Problématique

Nous nous sommes demandé si des caractéristiques précises comme le revenu des habitants, la taille de la ville ou le type de taxe pouvaient vraiment jouer un rôle déterminant dans la production et la répartition des déchets. Est-ce qu'une ville riche trie mieux ? Est-ce que la taxe incitative change vraiment les comportements ? Et pour le tri au quotidien, est-ce qu'une IA peut réellement aider à ne plus se tromper de poubelle ? Nous en sommes donc arrivés à cette problématique :

Comment l'intelligence artificielle peut-elle soutenir une gestion intelligente des déchets en combinant la prévision des flux communaux et l'assistance au tri pour les citoyens à partir d'images ?

1.4 Objectif

Cette question de recherche est essentielle car elle permet aux communes de mieux comprendre quels facteurs influencent le tri et de tester l'impact de leurs décisions (comme mettre en place la taxe) avant de les appliquer. Il y a aussi un intérêt pour les habitants qui peuvent utiliser l'IA pour apprendre à mieux trier, et cela de façon ludique. Pour répondre à cette problématique, nous avons fixé trois objectifs :

- Prédire les flux : Créer des modèles pour estimer la quantité de déchets et leur répartition par classe d'une ville selon différents facteurs.
- Aider au tri : Utiliser la reconnaissance d'images (CNN) pour identifier les types de déchets (plastique, verre, etc.) et dire où les jeter.
- Mettre en ligne : Créer un site web complet où il est possible de tester ces outils en temps réel grâce à une architecture connectant notre site à nos modèles d'IA.

Chapitre 2

2 Matériel et méthodes

2.1 Logiciels

2.1.a Logiciels et Environnements Technologiques

- Visual Studio Code (version 1.115.0) : Pour partager tous nos codes (en lien avec notre GitHub)
- GitHub (version Web 2026) : Répertoire du projet
- Discord (version 220.10) : Pour échanger rapidement entre membres du groupe.

2.1.b Sciences des Données et Intelligence Artificielle

- RStudio (version 2024.09.0) : Pour réaliser les modèles statistiques.
- Google Colab (version Cloud, Runtime 2026) : Pour entraîner les modèles CNN.
- Board : Pour trouver la base de données [1]
- Kaggle : Pour trouver la base de données [2]
- Gemini (version 3 Flash)

2.1.c Gestion des Données et Déploiement

- PhpMyAdmin/MAMP (version 5.2.2) : Pour importer la base de données, la trier et effectuer des requêtes SQL.
- Render : API
- InfinityFree : Pour héberger notre site.

2.1.d Bureautique et Multimédia

- LibreOffice Calc (version 25.2.2)
- Google Drive: Pour partager les fichiers du projet.
- Canva : Pour créer notre support vidéo.
- iMovie (version 10.4.3) : Pour le montage vidéo de notre soutenance.

2.2 Librairies

Manipulation et traitement de données :

- Pandas

Machine Learning :

- Scikit-Learn : Pour la séparation des données (*train_test_split*), l'entraînement de nos modèles de régression (*LinearRegression*, *RandomForestRegressor*) et le calcul des métriques de performance (*r2_score*, *mean_squared_error*, *confusion_matrix*)
- TensorFlow
- Keras

Site Web et API :

- FastAPI
- Uvicorn
- MySQL-connector-python
- rpy2

Analyses Statistiques et Modèles (RStudio) :

- Caret
- RandomForest
- Net

Calcul numérique et prétraitement des données :

- Numpy
- SimpleImputer (Sklearn)

Visualisation et cartographie :

- Matplotlib
- Seaborn
- Geopandas

2.3 Ordinateur utilisés

Nina :

Ordinateur utilisé : Acer Aspire Vero
Système d'exploitation : Windows 11
Processeur : Intel Core i5-1155G7 2.50 GHz
Mémoire RAM : 16 Go
Stockage : 512 Go SSD

Ambre :

Ordinateur utilisé : MacBook Pro
Système d'exploitation : mac OS Ventura 13.7.5
Processeur : 2,3 GHz Intel Core i5 double coeur
Mémoire RAM : 8 Go
Stockage : 500 Go

Anthony :

Ordinateur utilisé : HP Victus
Système d'exploitation : Windows 11 Famille
Processeur : Intel Core i5-12500H ou AMD Ryzen 5 5600H
Mémoire RAM : 16 Go DDR4
Stockage : 256 Go

Victor :

Ordinateur utilisé : MacBook Air 13”
Système d'exploitation : macOS Sequoia 15.5
Processeur : Puce Apple M4 (CPU 10 cœurs)
Mémoire RAM : 16 Go
Stockage : 256 Go

Chapitre 3

3 Bases de Données

3.1 Provenance des données

Lors de ce projet nous avons utilisé deux jeux de données trouver sur le site Board et sur le site Kaggle.

Le premier jeu de données *Municipal Waste Management Cost Prediction* (Di Foggia, 2022) [1] regroupe des informations statistiques détaillées sur la gestion des déchets de 4 341 communes italiennes. On y trouve alors des données croisées sur la démographie, l'économie locale, la situation géographique, les volumes de déchets produits par catégorie, ainsi que les coûts lier au traitement des déchets.

Le deuxième jeu de données, intitulé *Garbage Dataset Classification* (Cofone, Leonardo, 2025) [2]. Il regroupe une vaste collection de photographies de déchets ménagers, capturées sous différents angles et conditions de luminosité.

3.2 Descriptif des données

3.2.a Jeux de données 1 : Municipal Waste Management Cost Prediction

Ce jeu possède 39 colonnes correspondant aux différentes variables explicatives et cibles que l'on possède, réparties sur 4 341 lignes. Chaque ligne représente une municipalité unique, ce qui nous permet d'avoir une vision globale et diversifiée du territoire italien.

L'objectif est d'étudier la répartition en pourcentage de chaque grande catégorie de matériaux collectés. Sur ce jeux de données 9 classes sont représentées : Organique, Papier, Verres, Bois, Métal, Plastique, Équipements électriques et électroniques, Textile, et autre.

Voici les principales donné présent dans la table.

Nom colonne	Type	Signification	Caractéristique
region	Chaîne de caractère	région	
name	Chaîne de caractère	nom de la commune	
tc	Double	coût total de gestion des déchets par habitant	en €
area	Float	superficie	
pop	Nombre	nombre d'habitant	
finance	Float	revenus globaux de la municipalité	en €
plastic	Float	pourcentage de déchets plastiques collectés	%

3.2.b Jeu de données 3 : *Garbage Dataset Classification*

Ce jeux de données est dédié à l'entraînement de notre modèle de vision par ordinateur. On y trouve 13 901 images classées par catégories: Verre, Papier, Carton, Plastique, Métal, Ordures ménagères.

Ce jeu de données contient des images standardisées : redimensionnées à 256x256 pixels, au format RVB, et sans doublons. Ce jeu possède également un fichier csv contenant les étiquettes des images.

Type déchets	Carton	Verre	Métal	Papier	Plastique	Ordures ménagères
Quantité	2214	2500	2084	2315	2288	2500

Jeux de données 2 : "Garbage Dataset Classification" (11 841 images)

3.3 Modèles MCD et MOD

Afin de poser des bases solides pour le projet, nous avons débuté cette phase par l'élaboration d'un MCD, présenté ci-dessous. Ce travail de modélisation a été essentiel pour visualiser l'organisation des informations et définir précisément les relations entre les différentes entités. Nous avons créé la variable taux qui reprend tous les pourcentages de déchets de la base mais en fonction de l'id_materiau et de l'id_municipalite.

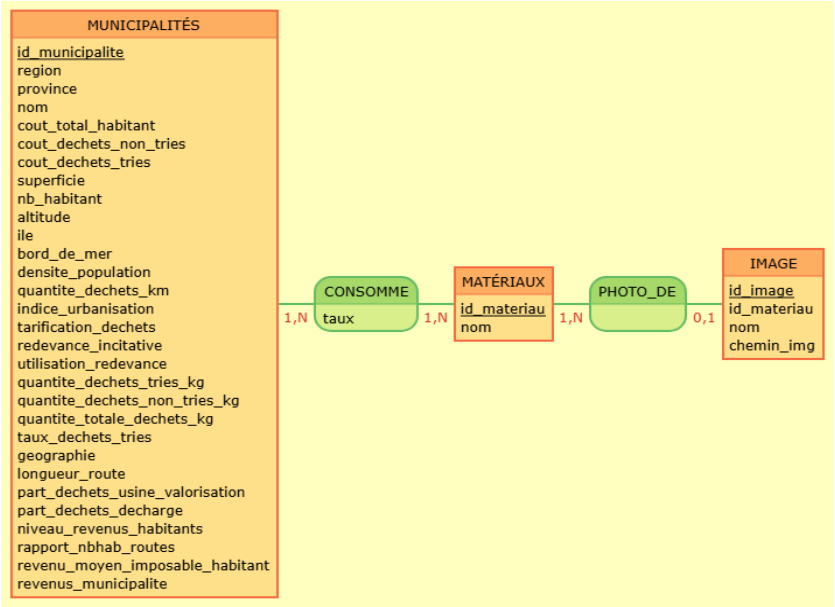

Figure 1: MCD Base de données 1 : "Municipal Waste Management Cost Prediction"

La principale difficulté a résidé dans la définition d'une méthode de classement efficace permettant de lier nos deux bases de données, puisque l'une d'entre elles est composée d'images. Pour surmonter cet obstacle et optimiser le stockage, nous avons pris la décision de conserver uniquement les chemins d'accès des images au sein de la base, plutôt que d'intégrer leurs données binaires. Ce choix technique a permis d'alléger considérablement la base de données tout en conservant une accessibilité fluide aux fichiers, le format binaire n'étant pas indispensable à ce stade de notre projet.

3.4 Nettoyage

Avant de lancer nos modèles, on a dû passer par une étape de nettoyage car les données brutes comportaient plusieurs problèmes qui auraient pu fausser nos résultats.

- Gestion des valeurs manquantes : Le fichier d'origine comportait de nombreux “trous” ou données non renseignées, ce qui constitue un obstacle majeur pour l'entraînement de modèles de régression. Nous avons donc choisi de remplacer ces valeurs manquantes par “0”, permettant ainsi de conserver l'intégralité du dataset sans supprimer quoi que ce soit.

- Uniformisation des noms : C'était l'un de nos plus gros défis techniques. Dans le fichier CSV, certains noms de régions étaient en italien, d'autres avec des accents différents, alors que notre carte SVG et notre base SQL utilisaient des noms standardisés. On a dû créer un script pour “mapper” (faire correspondre) chaque nom afin que la base de données puisse communiquer correctement avec l'interface web. Malgré cette normalisation, un bug subsiste sur la version finale de notre site concernant l'onglet de prédiction des communes. Nous avons identifié un problème de traitement concernant les régions qui contenaient un espace en leur nom. Actuellement, les deux seules régions concernées par cette particularité syntaxique sont les seules sur lesquelles il est impossible de générer des prédictions. Ce bug est probablement lié à la manière dont l'URL ou la requête SQL interprète les caractères d'espacement.

- Filtrage des valeurs aberrantes :

- Retrait de l'échelle logarithmique : Le dataset d'origine présentait des variables (comme le revenu) sous forme de logarithmes. Pour que nos analyses et notre interface soient compréhensibles par tous, nous avons transformé ces données en appliquant la fonction exponentielle directement dans le fichier.

Chapitre 4

4 Analyses exploratoire des données

L'objectif central de notre projet est de prévoir le taux de production de déchets par catégorie (organique, papier, verre, bois, métal, plastique, électrique/électronique, textile et autres) pour une commune donnée, en fonction de ses caractéristiques propres.

Avant de commencer à construire nos modèles de Machine Learning destinés à la prédiction, il a été cependant nécessaire de passer par une analyse d'exploration de nos données. Effectivement, la mise en place de graphiques nous a permis d'avoir une meilleure compréhension de la structure de nos données et a même commencé à mettre en évidence certaines corrélations.

Nous répertorions ici, l'intégralité des graphiques effectués durant cette phase, accompagnés de leurs analyses.

4.1 Visualisation

4.1.a Analyse du Jeu de Données 1 : *Municipal Waste Management Cost Prediction*

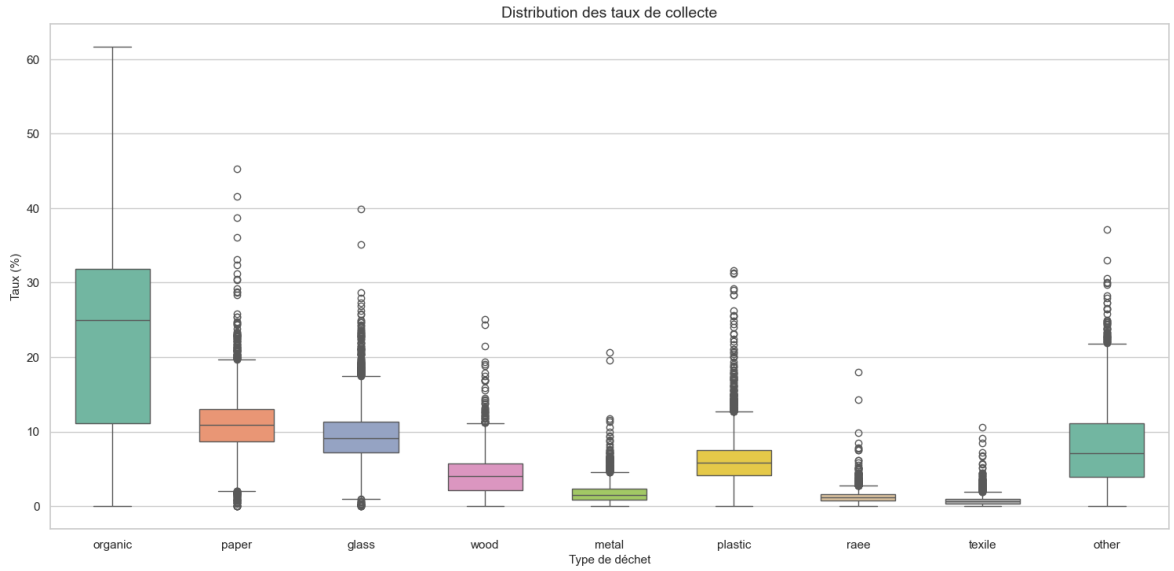

Figure 2: Distribution des taux de collecte par type de déchet.(légende)

Ce graphique représente le taux de déchets collecté de chaque aux communes en fonction des types de déchets. Les déchets organiques sont les plus collectés mais avec de fortes disparités. Le papier et le verre suivent avec une médiane autour des 10%. À l'inverse, le textile et l'électronique stagnent proche de 2%. Globalement, les déchets organiques sont les plus représentés et le reste des taux reste sous la barre des 10 %. On peut noter également que certaines communes obtiennent des résultats extrêmes bien au- dessus de la moyenne.

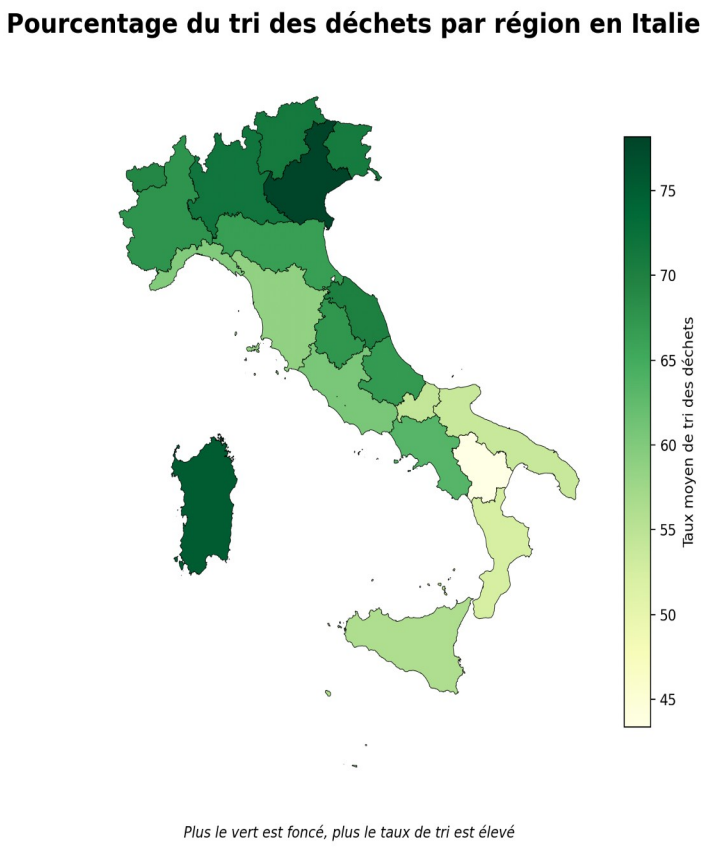

Figure 3: Carte régionale du taux de tri sélectif en Italie.

Ici une carte réalisée avec matplotlib (librairie vue pendant le 1er semestre de L3 en Science des Données) représentant le pays de l'Italie ainsi que ses 20 régions délimitées. On obtient une représentation par région du taux de tri des déchets, avec un affichage de plus en plus vert foncé si la ce taux est plus élevé dans la région défini. On peut ainsi remarquer qu'une tendance se dégage, avec un pourcentage de taux plus élevé au fur et à mesure que l'on se dirige au nord du pays, dans les régions plus riches. Cela confirme donc nos pensées du graphique en barres sur le taux de tri par régions. La carte est issue du site de l'Institut National de Statistique italien mettant à jour les frontières chaque année.

On compare la distribution des coûts totaux par habitant (€), selon deux systèmes de taxes des déchets (Initiative (PAYT - Pay As You Throw) et Fixe (Standard)).

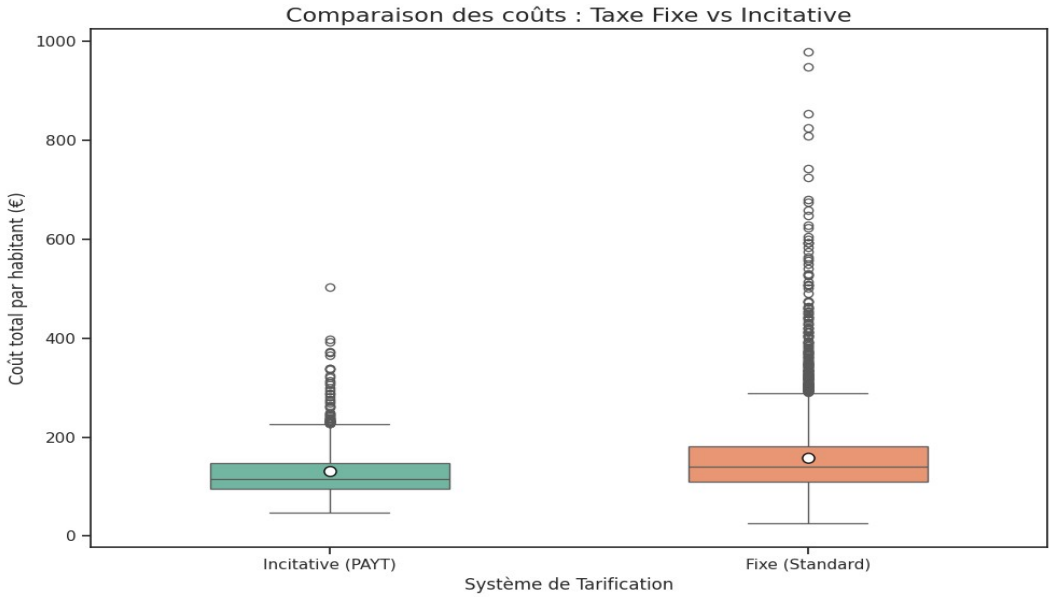

Figure 3: Comparaison du coût annuel par habitant entre la tarification incitative (PAYT) et la taxe fixe standard.

La taxe au poids (PAYT) permet de mieux maîtriser les coûts que la taxe fixe. La “boîte” est plus basse, ce qui prouve que le système est efficace pour la majorité des villes. Par contre, les quelques points extrêmes nous montrent que la taxe dépend d’autres variables aussi.

Ce graphique nous permet de comparer les 20 villes les plus productrices de déchets (ayant la variable “msw” la plus élevée). Les villes ont été regroupées par régions.

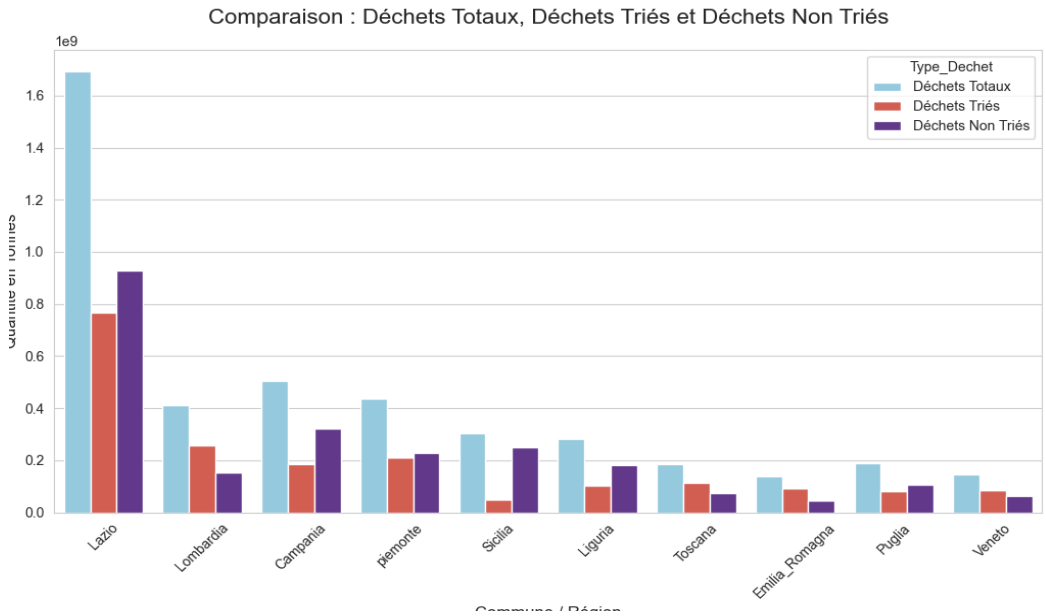

Figure 4: Comparaison des volumes de déchets totaux, triés et non triés pour les principales régions italiennes (en tonnes).

On observe que la Lazio est la région qui produit le plus de déchets, avec environ 1.7 milliard de tonnes. Une part importante de ces déchets est encore non triée, cela peut s’expliquer par le fait que Rome, la capitale, est dans la région de Lazio. Rome est la ville la plus peuplée d’Italie, et il y a beaucoup de tourisme dans cette ville. D’autres régions ont cependant une part plus importante de déchets triés comme Lombardia, Toscana, Emilia-Romagna et Veneto.

Nous avons décidés de faire une matrice de corrélation entre les variables. Cette matrice permis de détecter d'éventuels phénomènes de colinéarité et de mettre en évidence les variables ayant l'impact le plus significatif sur les performances de tri et les coûts de traitement.

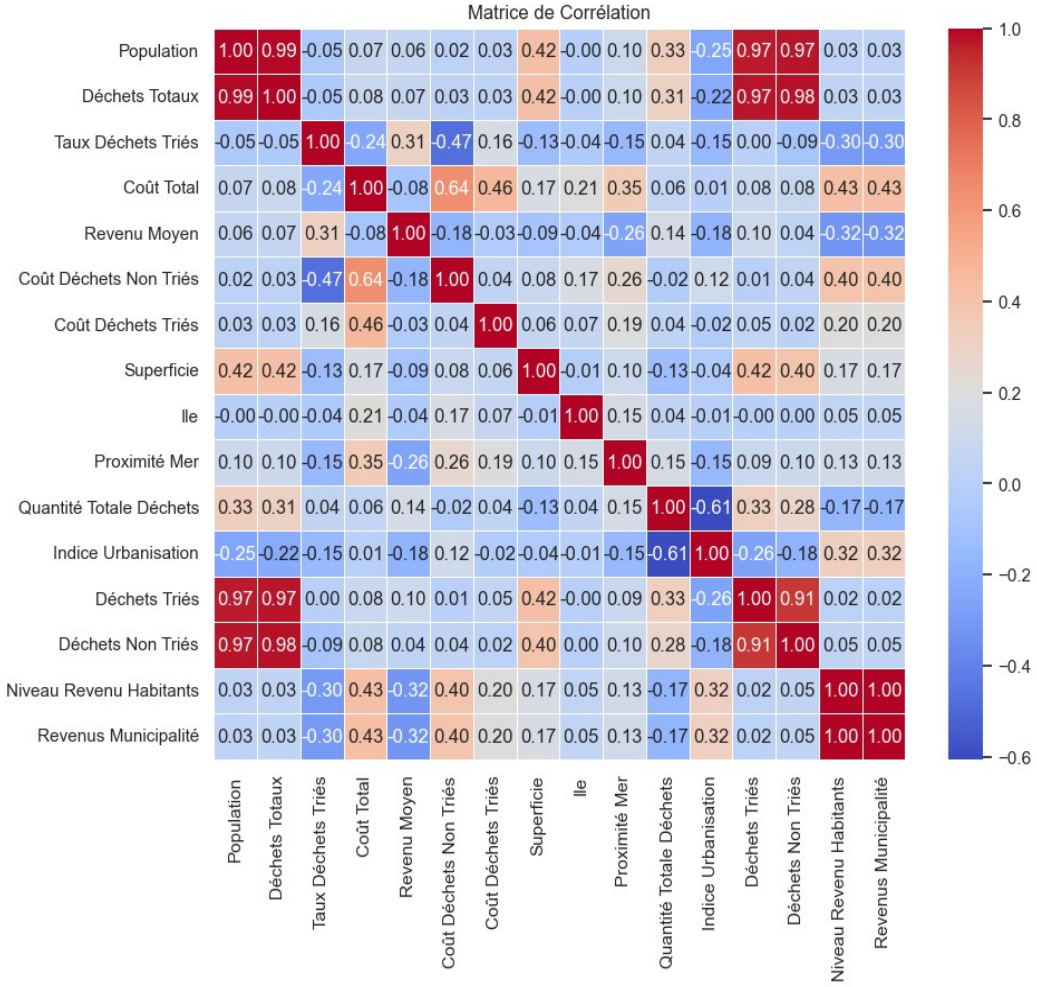

Figure 5: Matrice de corrélation entre les variables et les indicateurs de gestion des déchets.

Notre matrice de corrélation révèle que le volume total de déchets (msw) et le coût de gestion (tc) sont quasi-parfaitement corrélés à la population, mais que la véritable performance se mesure au taux de tri (sor). On observe une fracture nette selon le mode de tarification : les communes utilisant la taxe au poids (PAYT) affichent systématiquement des taux de recyclage supérieurs aux communes sous taxe STANDARD, prouvant que l'incitation financière surpasse la simple sensibilisation. Aussi, la morphologie urbaine joue un rôle clé : les zones à forte densité (wden et urb élevés) peinent davantage à trier efficacement,

probablement à cause des contraintes logistiques de l'habitat collectif. Enfin, l'analyse financière suggère que le coût total est souvent alourdi par les déchets non triés (msw_un), confirmant que le recyclage n'est pas seulement un impératif écologique, mais aussi une stratégie de réduction des coûts municipaux à long terme.

On compare la moyenne de chaque matériau pour voir ce que les gens mettent le plus dans les bacs de recyclage.

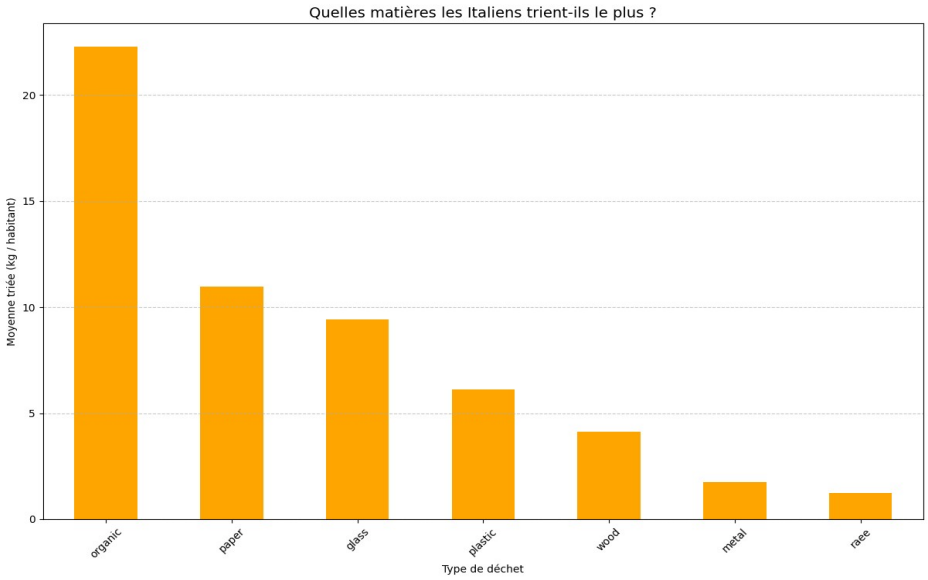

Figure 6: Quantité moyenne de déchets triés par habitant en Italie, ventilée par type de matériau.

Ce qu'on remarque tout de suite avec ce graphique, c'est que le tri n'est pas du tout équilibré entre les matériaux : c'est l'organique qui gagne haut la main. Comme c'est la matière la plus lourde, c’est elle qui fait monter les scores de recyclage. Le papier et le verre suivent loin derrière, tandis que le plastique semble faible alors qu'il prend beaucoup de place : c’est simplement parce qu’il est très léger en poids. En conclusion, l’analyse montre que le tri n’est pas qu’une question de bonne volonté, c’est une question de poids.

Ce graphique est appelé ”Bubble Chart”, il croise la taille de la population, le taux de recyclage, le volume total de déchets et le mode de facturation.

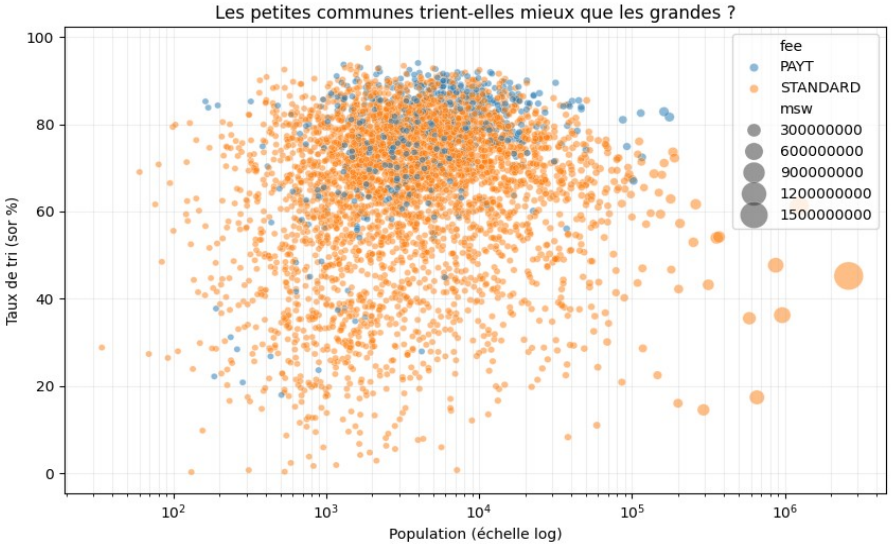

Figure 7: Corrélation entre la taille de la population (échelle logarithmique) et le taux de tri sélectif, différenciée par mode de tarification (PAYT vs Standard).

Il sert de représentation des déchets par commune en croisant, leur taille et leur mode de facturation, il nous permet de visualiser instantanément l’efficacité du recyclage à l’échelle d’un territoire. Malgré un nombre moins élevé, il y a une tendance montrant que les communes utilisant la taxe au poids (PAYT) affichent un meilleur taux de tri de déchets plus élevé que la taxe (STANDARD). Enfin, les villes les plus peuplées, celles représentées à droite, ont tendance à avoir un taux de tri plus bas, montrant la difficulté d’avoir un système de tri efficace à grande échelle.

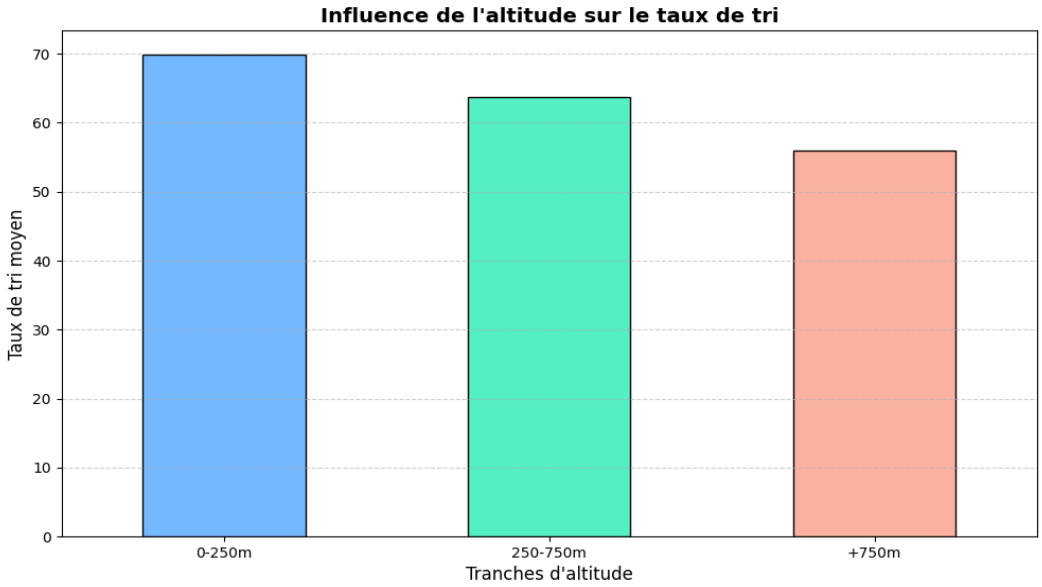

Figure 8: Influence de l'altitude sur le taux de tri

Ce graphique en barre étudie une possible corrélation entre l'altitude des communes et leurs efficacités en termes de tri.

Tout d'abord les communes ont été classées selon leurs altitudes en 3 catégories: - de 250 m, 250-750 m et + de 750 m. Une fois cela fait, on calcule pour chacune de ces catégories, la moyenne en % du taux de déchets triés. On peut supposer une corrélation donc entre ces deux facteurs : le taux de tri diminue à mesure que l'altitude augmente. De 0 à 250 m, le taux moyen atteint les 70% tandis qu'en montagne, le taux n'atteint "seulement" 56%. Cette tendance peut s'expliquer par plusieurs facteurs logistiques propres aux zones de haute altitude comme l'accessibilité (collecte plus complexe et coûteuse dans zones montagneuses, accès difficile pour les camion de ramassage), les infrastructures de tri (centres de tri et déchetteries situées plus loin des villages de hautes montagnes que des villes de plaine) où encore la saisonnalité (zones touristiques de montagnes subissent des pics de fréquentation saisonnière qui rend la gestion des déchets plus difficile).

Chapitre 5

5 Machine Learning

5.1 Base de données 1 : Prédictions communales

Pour la base de données *Municipal waste management*, nous avons décidé d'entraîner des modèles comme la Régression Linéaire, la Régression Logistique Multinomiale et le Random Forest.

Notre objectif est de prédire la répartition des déchets en fonction de variables explicatives.

Nous avons également fait des comparaisons entre la régression linéaire et le Random Forest, ainsi qu'entre la régression logistique multinomiale et le Random Forest.

5.1.a Régression Linéaire Multiple

La qualité d'une régression dépend énormément de la propreté des données en entrée. N'ayant pas de certitude absolue sur la méthode de traitement des données manquantes la plus adaptée à ce jeu de données, nous avons choisi de tester deux stratégies distinctes :

- **Variante par Imputation (Moyenne) :** Nous remplaçons chaque valeur manquante par la moyenne de sa colonne respective. Cette méthode permet de conserver l'intégralité des observations (lignes) du dataset, au risque de réduire artificiellement la variance des variables.
- **Variante par Suppression :** Nous supprimons systématiquement toute ligne contenant au moins une valeur manquante. Cette approche garantit que le modèle ne s'entraîne que sur des données réelles et non extrapolées, bien qu'elle réduise la taille de notre échantillon d'entraînement.

Comparer ces deux modèles nous permettra de déterminer si la perte de volume de données (suppression) est plus importante à la précision du modèle que l'introduction d'un biais statistique (moyenne).

La régression linéaire multiple est une technique statistique qui modélise la relation entre une variable numérique à prédire et plusieurs variables explicatives. Elle permet non seulement de prédire un résultat, mais aussi d'isoler l'effet spécifique de chaque variable en gardant les autres constantes. Nous avons aussi étudié ce modèle en cours, le fait de pouvoir le mettre en œuvre dans ce projet nous intéressait d'autant plus.

Dans le cadre de notre étude comparative des modèles d'IA pour la gestion des déchets, la Régression Linéaire sert de référence pour notre équipe : elle nous permet de comprendre si les variables comme la population ou le PIB ont une influence sur la proportion des types de déchets. Pour commencer, nous avons choisi d'appliquer une régression linéaire distincte pour chaque classe afin de modéliser plus précisément les relations propres à chacune d'elles.

5.1.b Random Forest

Le modèle Random Forest est un algorithme d'apprentissage par ensemble (Ensemble Learning). Au lieu de s'appuyer sur un seul estimateur, il génère une multitude d'arbres de décision indépendants entraînés sur des sous-ensembles de données. Chaque arbre produit sa propre estimation, et le résultat final est obtenu par agrégation : pour des valeurs numériques (régression), l'algorithme calcule la moyenne de toutes les prédictions. Cette approche par "vote majoritaire" permet de compenser les erreurs individuelles de chaque arbre, offrant ainsi un modèle hautement fiable, stable et robuste face au bruit statistique.

Initialement, nous avions conçu un modèle unique "multi-sorties" pour prédire simultanément les neuf catégories de déchets. Cependant, l'hétérogénéité de notre dataset entraînait la suppression automatique de toute ligne dès qu'une seule valeur de classe manquait, réduisant grandement la taille de notre échantillon.

Pour pallier cette dispersion, nous avons opté pour une architecture de modèles experts, plus évolué. En isolant chaque catégorie de déchets, nous avons optimisé l'apprentissage pour chaque flux. La stratégie d'entraînement a été déclinée en trois scripts (deux en Python, un en R) pour tester différentes gestions des données manquantes :

- Approche par suppression : Une observation n'est conservée que si la catégorie de déchet et l'intégralité des variables explicatives sont renseignées, garantissant un apprentissage sur des données réelles et complètes.

- Approche par imputation : Les valeurs manquantes sont remplacées par la moyenne de la variable, permettant de conserver un volume de données plus important.

Le script R a été fait pour permettre au Random Forest de se perfectionner via l'ajout de la variable qualitative "region" grâce à l'outil "as.factor", non disponible sur Python. Cela permettait aussi de comparer ce modèle à celui de la régression logistique multinomiale.

b) Description de l'architecture et des variables

Le modèle final repose sur une forêt dense dont les paramètres ont été ajustés pour maximiser la précision tout en évitant le surapprentissage (overfitting) :

- n_estimators = 500 : Stabilise la prédiction via un grand nombre d'arbres.
- max_depth = 10 : Limite la complexité pour ne capturer que les tendances structurelles.
- min_samples_split=10 et min_samples_leaf = 5 : Assurent que chaque règle repose sur un groupe minimal de communes.
- max_features = 0.5 : Force l'exploration de diverses combinaisons de variables.

Le succès du modèle repose sur l'utilisation de 10 variables clés (densité, salaire moyen, altitude, etc.), la variable "Région" s'étant révélée être la variable maîtresse du script R. Nous avons utilisé pour le script R, la suppression totale des données manquantes qui offrait de meilleurs résultat que celle par imputation de la moyenne. Les paramètres y ont été un peu modifiés par rapport a la version Python, il était plus simple mais plus précis.

Pour évaluer la précision, nous avons utilisé deux indicateurs de performance :

- Le score R2 (Coefficient de détermination) : Il mesure la capacité du modèle à expliquer la variance des données (plus il est proche de 1, plus le modèle est performant).
- Le score NRMSE (Erreur Quadratique Moyenne Normalisée) : Il exprime l'écart moyen entre les prédictions et les valeurs réelles, normalisé pour permettre la comparaison entre différentes échelles de déchets.
- Le score MAE (Erreur Absolue Moyenne) : utilisé seulement pour le script R, il représente la moyenne des écarts absolus entre les prédictions et les valeurs réelles.

Il exprime l'erreur dans l'unité physique de la donnée: plus il est proche de 0, plus la prédiction est précise.

Enfin, le calcul de l'importance des variables nous a permis d'identifier quels facteurs (socio-économiques ou géographiques) étaient les plus explicatifs pour chaque flux de déchets.

5.1.c Régression Logistique Multinomiale

La régression logistique multinomiale constitue l'extension naturelle de la régression logistique classique. Alors que la version standard répond à une logique binaire (Oui/Non), le modèle multinomial permet de prédire l'appartenance à plusieurs catégories mutuellement exclusives, comme les différents types de matériaux (Papier, Verre, Plastique, etc.). Mathématiquement, l'algorithme choisit une catégorie de référence et estime, pour chaque autre catégorie j, une équation linéaire représentant la “force” d'appartenance à celle-ci. Ces scores réels sont ensuite transformés en pourcentages exploitables via la fonction Softmax. Pour ajuster les coefficients, le modèle utilise le maximum de vraisemblance, une méthode itérative qui fait correspondre les probabilités prédites aux observations réelles du jeu de données.

La mise en place de ce modèle a nécessité une préparation rigoureuse des données. Nous avons d'abord traité les valeurs manquantes en les remplaçant par 0, une étape indispensable puisque l'algorithme ne peut calculer de pourcentages sur des vides. Pour garantir la puissance de l'apprentissage, nous avons appliqué une validation croisée stratifiée en 5 plis (k=5) avec la fonction *createFolds* du package *caret*. Cette stratification par région est importante car elle assure que chaque zone géographique est représentée au moins cinq fois, évitant ainsi que le script ne plante ou ne produise des prédictions sur des territoires inconnus.

Une des difficultés a été la sur-représentation de la catégorie *Organic*. Sans modification, le modèle cherchait à minimiser son erreur globale en prédisant systématiquement cette classe dominante. Pour corriger ce biais, nous avons limité cette catégorie à 200 lignes, forçant ainsi l'algorithme à mieux apprendre les caractéristiques des autres matières. Enfin, plutôt que de calculer une droite classique, l'algorithme *multinom* (package *nnet*) traite 9 catégories simultanément et produit un vecteur de probabilités pour chaque ville.

Pour valider la fiabilité de nos prédictions, nous avons calculé un intervalle de confiance à 95% sur l'erreur moyenne absolue (MAE). Cette approche permet de s'assurer que la précision affichée n'est pas le fruit d'un échantillonnage favorable, mais bien une caractéristique du modèle.

Nous avons choisi des variables qui nous permettent de prédire le taux de déchets de chaque classe. Nous avons des variables qui nous informent sur la population, la richesse et sur la structure. Pour les informations sur la population, nous utilisons _pop, pden_ et _urb_ (respectivement, le nombre d'habitant, la densité et l'indice d'urbanisation), qui va aider le modèle à savoir si nous sommes dans une métropole dense (plus d'emballages, de plastique et de papier) ou une zone rurale (potentiellement plus d'organique). Sur le plan financier, nous utilisons _gdp, wage_ et _finance_ (respectivement, le niveau de revenu par habitant, le revenu moyen imposable par habitant et les revenus de la municipalité), qui peut nous indiquer si certains déchets plus rares ou technologiques sont présents. Enfin, pour les facteurs géographiques, nous utilisons _alt, sea_ et _region_ (respectivement, l'altitude, la présence de bord de mer et la région), qui nous aide à connaître les contraintes géographiques comme le tourisme.

5.2 Base de données 2 : Reconnaissance d'images (CNN)

5.2.a Introduction

Pour prédire le type de déchets a partir d'une image, nous avons décidé de créer et d'entraîner un réseau de neurones convolutif à partir de notre base de données d'images. Les réseaux de neurones convolutif, dont Yann LeCun développa le premier réseau fonctionnel (1998) [3], en s'appuyant sur les travaux de D. H. Hubel et T. N. Wiesel (1968) [4] sur le cortex visuel du chat et des primates. Révolutionné par le modèle Alexnet en 2012 [5], les réseaux de neurones convolutifs sont aujourd'hui devenus la méthode principale en classifaction d'image.

Leur plus gros avantage est qu'ils aprennent eux-mêmes à identifier les caractéristiques importantes des images via les couches. Les réseaux de neurones convolutifs sont également invariants à la translation et conservent les relations spatiales.

Pour résumer, les réseaux de neurones convolutifs se composent de 3 types de couches :

- Une couche de convolution qui agit comme un filtre sur l'image pour extraire des caractéristiques (ligne verticale, horizontale, couleurs) via des opérations matricielles.
- Une couche de pooling qui réduit la taille des images tout en gardant les informations importantes pour accélérer le traitement. On réduit les zones en prenant généralement la moyenne des pixels ou le maximum d'une zone donnée.
- Une couche totalement connectée qui prend les caractéristiques extraites par les couches précédentes. Et donne la décision finale de la prédiction.

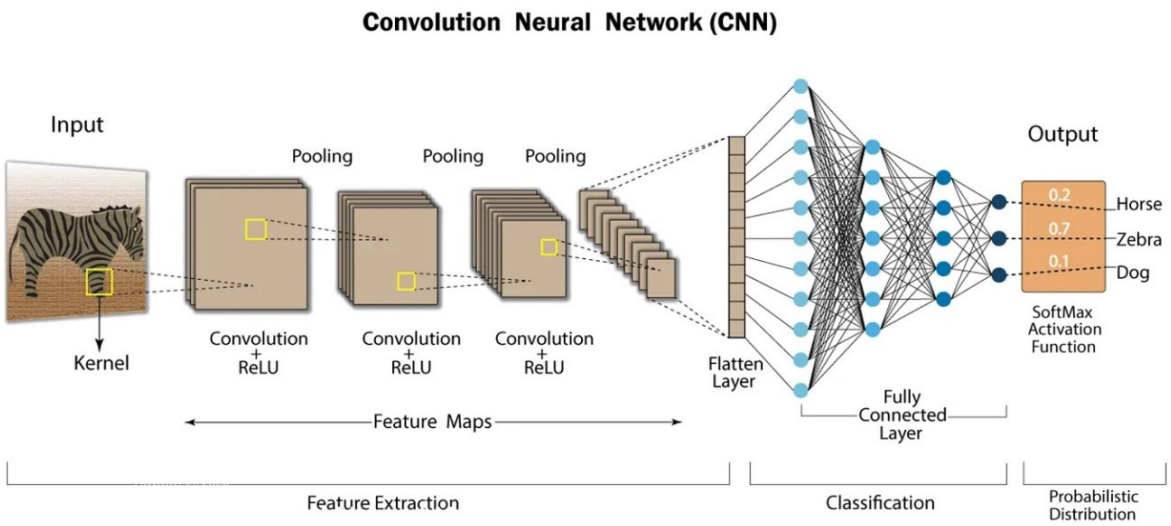

Figure 9: HAQUE, Kh Nafizul. "What is Convolutional Neural Network — CNN (Deep Learning)", LinkedIn, 3 avril 2023

Un CNN fonctionne en deux étapes. Premièrement il enchaîne des couches de convolution et de pooling pour réduire les données. Puis ces informations sont aplaties et transmises à la couche totalement connectée qui prend la décision finale en renvoyant une probabilité.

Pour notre projet nous possédons déjà une base de données avec des images à une taille identique (256 x 256). Nous avons également deux dossiers distincts : un avec des images pour entraîner le modèle et l'autre avec d'autres images pour tester le modèle.

Notre but est donc de prédire le type de déchets d'une image quelconque parmi les classes suivantes : Carton, Verre, Métal, Papier, Plastique et Ordures ménagères.

5.2.b Méthodologie de comparaison des modèles

Afin de comparer de manière rigoureuse et équitable les différents modèles de réseaux de neurones convolutifs, nous avons mis un protocole expérimental contrôlé.

Protocole expérimental

Tous les modèles sont entraînés dans des conditions identiques afin d'assurer la comparabilité des résultats. Les éléments suivants sont fixés (Figure 27):

- Même jeu de données et mêmes partitions entre les ensembles d'entraînement de validation et de test
- Mêmes hyperparamètres principaux : le nombre d'epoch est fixé à 20, le batch size (nombre d'image par paquets) est fixé à 32, l'optimiseur choisi est Adam, learning rate
- Même environnement matériel, pour rappelle tous les tests ont été réalisé via Google Colab avec le T4 GPU pour des raisons de facilité au sein du groupe et équité dans la comparaisons des modèles.

Afin de prendre en compte la variabilité dû aux facteurs aléatoires (initialisation des poids, ordre des données), chaque modèle est entraîné sur une même seed aléatoire.

Les seeds utilisées sont identiques pour tous les modèles, ce qui permet une comparaison équitable.

Nous avons donc retenu une moyenne et un écart-type comme outils d'analyse pour chaque modèle. Cette approche permet de valider la performance du modèle tout en s'assurant de sa stabilité**.**

Métriques d'évaluation

Pour comparer les performances de ces modèles nous allons utiliser plusieurs critères d'évaluation :

- Accuracy (%)sur le jeu de test. Elle est la métrique principale de comparaison
- Temps d’entraînement
- Nombre de paramètres

5.2.c Description des Architectures

Pour nos modèles, nous avons utilisé la fonction d'activation ReLU.

La fonction d’activation permet au réseau de neurone convolutif d’apprendre des relations complexes en introduisant de la non-linéarité après chaque couche.

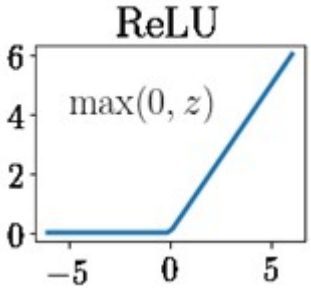

Figure 10: Fonction ReLu

La fonction d’activation ReLU présente plusieurs avantages. Elle permet d’accélérer l’apprentissage grâce à sa simplicité de calcul et en évitant que les gradients deviennent trop faibles. Enfin, elle rend le réseau plus efficace en supprimant les valeurs négatives, ce qui crée des neurones inactifs et simplifie l’apprentissage.

Pour tous les modèles nous avons utilisé l'optimisateur Adam.

Un optimiseur est un algorithme mathématique qui ajuste les poids d’un réseau de neurones en utilisant le gradient de la fonction de perte, afin de minimiser l’erreur du modèle. La valeur par défaut du taux d’apprentissage d'Adam est de 0.001.

Pour cela, on calcule le gradient, puis on applique une méthode de descente de gradient. Le gradient indique la direction et l'intensité de la plus forte pente de l'erreur. Puis l'optimiseur met à jour les poids en soustrayant une fraction du gradient aux valeurs actuelles.

Adam est une version améliorée de la descente de gradient. Il utilise une estimation des moments du gradient afin de converger plus efficacement vers un minimum de la fonction de perte.

Modèle avec couches en décroissant :

Pour notre premier modèle, nous avons chosi une structure en entonnoir avec un nombre de filtres décroissants (128 → 64 → 32 → 16).

L'idée étant de capturer un grand nombre de détails au début, puis dissiper cette information pour ne garder que les caractéristiques les plus discriminantes avant la classification (Figure 28).

Pour commencer, nous avons appliqué une normalisation des pixels afin de ramener les valeurs entre 0 et 1, ce qui facilite la convergence du modèle.

Concernant les couches de convolution, nous avons utilisé des noyaux de taille 4×4.

Le noyau est une matrice qui se déplace sur l'image afin d'extraire des caractéristiques locales. Une taille de noyau de 4 permet de capter des motifs relativement larges.

Après les couches de convolution et de pooling, nous utilisons une couche d'applatissement.

Cette couche transforme les matrices de caractéristiques en un vecteur 1D. Cela permet de passer des features spatiales à un vecteur exploitable par des couches entièrement connectées.

Cette dernière est composée de 64 neuronnes et de la fonction d'activation ReLu.

Son rôle est de combiner les caractéristiques extraites précédamment pour apprendre des relations plus globales entre elles.

Comme expliquer précédemment, la fonction ReLu permet d'ajouter de la non-linéarité au modèle pour apprendre des combinaisons complexes.

Entre les couches de convolution, nous avons ajouté des couches de pooling. Celles-ci permettent de réduire la dimension spatiale des cartes de caractéristiques, tout en conservant les informations les plus pertinentes.

Dans notre cas, nous utilisons le MaxPooling, qui consiste à conserver la valeur maximale de chaque noyau.

Pour la couche de classification nous avons choisi la fonction *softmax*.

Celle-ci transforme les sorties du réseau en probabilités. L'ensemble des sorties forme une distribution dont la somme est égale à 1.

Cela permet d'interpréter la sortie du modèle comme une répartition de probabilités sur les différentes classes, et ainsi de sélectionner la classe la plus probable.

La fonction de perte utilisée pour ce modèle est *SparseCategoricalCrossentropy.*

La cross-entropy mesure l'écart entre les probabilités prédites par le modèle et les vraies classes.

Le paramètre *from_logits = False* signifie que les sorties du modèle sont déjà transformées en probabilités grâce à la fonction softmax.

Modèle avec couches en croissant

Pour ce deuxième modèle la structure est exactement la même excepté le nombre de filtre, qui est ici croissant. Ce modèle suit une approche plus classique de vision par ordinateur, où la complexité augmente avec la profondeur.

Il utilise 4 blocs de convolution suivis d'une couche de MaxPooling. Le nombre de filtres double à chaque étape (16→32→64→128). Augmenter le nombre de filtre au fur et à mesure permet de capturer des motifs de plus en plus complexes (Figure 29).

Modèle AlexNet

Le réseau de neurones convolutif AlexNet, proposé en 2012 par Alex Krizhevsky avec Ilya Sutskever et Geoffrey Hinton [5], constitue une avancée majeure en vision par ordinateur. Il s'est dévoilé en remportant le concours ImageNet Large Scale Visual Recognition Challenge 2012. Ce réseau présente une nette amélioration des performances par rapport aux méthodes précédentes. AlexNet a marqué le début de l'essor du deep learning en démontrant l'efficacité des réseaux de neurones profonds pour la classification d'images.

Ce modèle repose sur une architecture composée de couches de convolution et de couches entièrement connectées. Il introduit plusieurs innovations importantes, notamment l'utilisation

de la fonction d'activation ReLU n'étant pas utilisé avant, l'entraînement sur des processeurs graphiques(GPU), ainsi que des nouvelles techniques comme le dropout et l'augmentation de données.

L'architecture d'AlexNet comporte cinq couches convolutionnelles suivies de trois couches entièrement connectées. Les couches convolutionnelles utilisent successivement : Conv1 (96 filtres , noyau 11×11, stride 4), Conv2 (256 filtres, noyau 5×5), Conv3 (384 filtres,noyau 3×3), Conv4 (384 filtres, noyau 3×3) et Conv5 (256 filtres, noyau 3×3), chacune suivie de ReLU et de max-pooling pour réduire la dimension spatiale. Les trois couches denses de 4096 neurones chacune appliquent ReLU et dropout. La dernière couche utilise softmax pour la classification (Figure 30).

Dans la première couche convolutionnelle on remarque que *stride = 4*. Cela signifie que le filtre 11×11 se déplace de 4 pixels à chaque pas sur l'image.

Le Dropout est une technique pour éviter le sur-apprentissage. Avec Dropout(0.5), 50% des neurones d'une couche dense sont désactivés au hasard à chaque itération d'entraînement. Cela force le réseau à ne pas dépendre de neurones spécifiques et à mieux généraliser.

Modèle VGGNet

Le modèle VGGNet a été présenté en 2014 par Karen Simonyan et Andrew Zisserman [6] de l'Université d'Oxford dans le cadre du challenge ImageNet 2014. Le réseau se distingue par sa simplicité et sa profondeur, utilisant uniquement des couches convolutionnelles 3×3 empilées et des couches max-pooling pour réduire progressivement la taille spatiale des images. VGGNet a démontré que augmenter la profondeur d'un CNN tout en conservant de petites tailles de filtre pouvait améliorer significativement la précision en classification d'images.

L'architecture de VGGNet que nous avons implémentée (Figure 31) se compose de cinq blocs convolutionnels, chacun utilisant uniquement des couches Conv2D 3×3 empilées avec ReLU, suivies d'un max-pooling 2×2. Les blocs utilisent successivement 64, 128, 256, 512 et 512 filtres. Les trois couches entièrement connectées finales comportent 4096, 4096 et le nombre

de classes, avec ReLU et Dropout 0.5. L’entrée est mise à l’échelle avec Rescaling, et le modèle est entraîné sur les données augmentées avec l’optimiseur Adam.

Après avoir constaté que les premiers modèles manquaient de précision, nous avons choisi d'intégrer un jeu de données plus volumineux [2] afin d'améliorer la fiabilité des résultats. Nous avons réutilisé les mêmes modèles afin de tester leur robustesse sur un échantillon plus représentatif.

Chapitre 6

6 Analyse et Résultats

6.1 Base de données 1 : Machine Learning

6.1.a Régression Linéaire Multiple

Pour évaluer l'impact du prétraitement sur la performance de notre Régression Linéaire Multiple, nous avons confronté deux approches : l'imputation par la moyenne et la suppression des données incomplètes.

DÉCHET	Lignes	R²	RMSE (%)
[organic]	Lignes: 3829	R²: 0.26	RMSE: ±48.87%
[paper]	Lignes: 4316	R²: 0.13	RMSE: ±32.16%
[glass]	Lignes: 4308	R²: 0.09	RMSE: ±38.94%
[wood]	Lignes: 3246	R²: 0.21	RMSE: ±56.55%
[metal]	Lignes: 4095	R²: 0.01	RMSE: ±76.68%
[plastic]	Lignes: 4302	R²: 0.07	RMSE: ±49.37%
[raee]	Lignes: 4027	R²: 0.02	RMSE: ±69.85%
[texile]	Lignes: 3328	R²: 0.03	RMSE: ±75.22%
[other]	Lignes: 4205	R²: 0.19	RMSE: ±60.73%

Figure 11: Résultat du modèle par imputation de la moyenne des données manquantes - Régression Linéaire Multiple

Modèle 1 Imputation par la moyenne : On remplace les données manquantes par la moyenne de la colonne. L'idée est de garder un maximum d'informations pour que le modèle prenne un maximum de communes différentes.

Un échantillon plus large réduit la variance des estimateurs de la régression.

Performance (R^2) : On obtient ici les meilleurs scores, notamment sur l'organique (0.26) et le bois (0.21). Cela prouve que même avec des valeurs moyennes injectées, le modèle apprend mieux les tendances globales.

Modèle 2 Suppression : On supprime toutes les lignes incomplètes. Ici, on privilégie la pureté des données : le modèle n'apprend que sur des chiffres réels, même si cela réduit drastiquement la taille de l'échantillon.

DÉCHET	Lignes	R^2	RMSE (%)
[organic]	3430	0.24	± 50.56%
[paper]	3877	0.11	± 32.80%
[glass]	3870	0.09	± 37.91%
[wood]	3002	0.16	± 58.44%
[metal]	3697	0.13	± 73.89%
[plastic]	3866	0.03	± 51.53%
[raee]	3629	0.04	± 56.11%
[textile]	3005	0.01	± 89.82%
[other]	3784	0.14	± 59.10%

Figure 12: Résultat du modèle par suppression des données manquantes - Régression Linéaire Multiple

Toute ligne comportant au moins une donnée manquante est exclue. On ne travaille qu'avec des données 100% réelles. Si cette méthode garantit une grande précision théorique, elle réduit notre base d'apprentissage (perte de plus de 1300 lignes pour certains déchets). On court alors le risque d'un biais de sélection, où le modèle n'apprend que sur les communes les mieux organisées , ignorant potentiellement la réalité des communes plus petites ou moins structurées.

Cette approche privilégie la pureté des données en n'utilisant que des observations complètes. Cependant, elle entraîne une perte massive d'informations, avec une réduction de l'échantillon allant jusqu'à 30% pour certains flux.

La distribution des données manquantes n'est pas aléatoire. Les communes exclues sont souvent celles disposant de moins de ressources administratives ou techniques pour le reporting. Les supprimer revient à limiter l'étude aux collectivités les plus structurées, ce qui occulte une partie de la réalité du terrain.

Dégradation de la précision : On observe une baisse systématique du R^2 (ex: passage de 0.26 à 0.24 pour l'organique). Cette perte de données limite la capacité d'apprentissage de l'algorithme.

Avec un échantillon réduit, le modèle devient plus vulnérable aux valeurs aberrantes, rendant les prédictions moins fiables pour les communes non présentes dans le jeu d'entraînement.

Pour notre projet, il est préférable de conserver l'approche par imputation (moyenne) pour la plupart des flux. Elle offre un meilleur score R^2 et évite de perdre jusqu'à 30% de nos données, ce qui rendra notre comparaison finale avec les autres modèles plus robuste sur un grand échantillon.

Au vu des scores obtenus, l'Imputation par la moyenne semble être la stratégie la plus équilibrée. Elle maintient un R^2

plus élevé pour les flux principaux (Papier, Verre, Bois) tout en évitant une perte massive d'observations, ce qui est crucial pour que notre régression serve de base solide à la comparaison avec les autres modèles de l'équipe.

6.1.b Random Forest

Les 3 modèles réalisés avec le modèle de Machine Learning “Random Forest” ont affichés des résultats différents mais intéressants.

DÉCHET	R²	NRMSE (%)	VARIABLE MAITRESSE
[organic]	0.45	± 43.3%	pop (46.2%)
[paper]	0.20	± 31.1%	wage (18.1%)
[glass]	0.21	± 35.4%	pop (22.0%)
[wood]	0.22	± 56.2%	wage (34.1%)
[metal]	0.22	± 69.9%	wage (21.1%)
[plastic]	0.08	± 50.3%	alt (16.6%)
[raee]	0.05	± 55.7%	wage (17.7%)
[textile]	0.03	± 88.7%	tc (17.3%)
[other]	0.29	± 53.5%	wage (24.6%)

Figure 13: Résultat du modèle par suppression des données manquantes -Random Forest

On remarque que les flux "prévisibles" (Organique, Verre, Papier) sont les mieux captés avec des R2 atteignant 0,45. À l'inverse, les flux "atypiques" (Textile, RAEE) restent difficiles à modéliser (NRMSE proche de 90%), car ils dépendent de facteurs non financiers comme la proximité de bornes de dons ou des habitudes de tri locales.

DÉCHET	LIGNES	R²	NRMSE (%)	VARIABLE MAITRESSE
[organic]	3829	0.47	± 41.1%	pop (46.7%)
[paper]	4316	0.18	± 31.2%	wage (17.1%)
[glass]	4308	0.23	± 35.8%	pop (23.1%)
[wood]	3246	0.26	± 54.6%	wage (35.0%)
[metal]	4095	0.17	± 70.2%	wage (18.7%)
[plastic]	4302	0.13	± 47.6%	alt (15.9%)
[raee]	4027	0.03	± 69.6%	wage (18.4%)
[textile]	3328	-0.01	± 76.5%	tc (15.7%)
[other]	4205	0.32	± 55.6%	wage (24.5%)

Figure 14: Résultat du modèle par imputation de la moyenne des données manquantes - Random Forest

Sachant que le modèle Random Forest augmente en efficacité si il est alimenté de plus données. Nous nous attendions à des résultats légèrement meilleurs avec le script d'imputation par la moyenne, ce qui fût le cas.

Les paramètres restent les mêmes. Les résultats restent assez semblables entre les deux algorithmes même si le modèle avec imputation de la moyenne peut se dégager comme étant légèrement supérieur, il est plus robuste en raison d'un nombre de données d'entraînement plus élevées.

--- Entraînement Random Forest (AVEC MAE & RÉGIONS) ---

DÉCHET	LIGNES	R²	MAE	NRMSE (%)
[organic]	3430	0.53	6.72	±39.45 %
[paper]	3877	0.26	2.30	±29.63 %
[glass]	3870	0.27	2.24	±33.25 %
[wood]	3002	0.35	1.56	±48.50 %
[metal]	3697	0.29	0.73	±61.82 %
[plastic]	3866	0.28	2.02	±45.41 %
[raee]	3629	0.04	0.49	±70.41 %
[textile]	3005	0.04	0.44	±87.44 %
[other]	3784	0.41	2.80	±48.14 %

Figure 15: Résultat du modèle ,avec ajout des régions, par suppression des données manquantes - Random Forest

L'ajout de la variable qualitative "region" a été un vrai plus pour notre modèle Random Forest. En effet, les résultats furent nettement supérieur. La stratégie utilisé ici est, pour rappel, la suppression des données manquantes.

L'intégration de la dimension régionale dans notre modèle R a transformé la précision de nos prédictions. Pour les déchets organiques, le coefficient de détermination (R2) atteint désormais 0.53, confirmant que la localisation géographique est un facteur aussi déterminant que le niveau de richesse.

Le gain le plus notable concerne le plastique, dont la prédictibilité a plus que doublé. Cela démontre que les politiques de gestion des déchets sont territoriales : à profil socio-économique identique, le pourcentage varie selon les infrastructures et les cultures régionales. Seuls le textile et les déchets électroniques restent difficiles à modéliser, confirmant leur caractère aléatoire et moins lié aux variables macro-économiques classiques.

L’ajout de la dimension géographique via les régions a permis de capturer des disparités locales que les chiffres seuls (PIB, population, etc..) ne reflétaient pas.

En conclusion, le modèle Random Forest peut être considéré comme satisfaisant sur certaines classes, ces dernières dépendent des données démographiques et économiques dont nous disposons. D’autres sont plus aléatoires et nécessitent d’autres informations que notre dataset ne dispose pas pour les expliquer.

6.1.c Régression Logistique Multinomiale

L'évaluation du modèle s'appuie sur deux métriques complémentaires : le MAE (écart moyen en points de pourcentage) et le RMSE, plus sévère car il pénalise lourdement les erreurs importantes. L'analyse des résultats montre que les catégories rares comme le Textile (MAE 0,94) et le RAEE (MAE 1,23) affichent les erreurs les plus faibles, témoignant d'une grande précision sur les faibles volumes. À l'inverse, l'erreur la plus forte concerne l'Organique (MAE 8,94), ce qui s'explique par la difficulté de prédire des comportements humains imprévisibles comme le compostage individuel.

Pour la plupart des classes, le RMSE est proche du MAE. Cela indique qu'il y a peu de "valeurs aberrantes". Le modèle est constant et ne fait pas d'erreurs catastrophiques sur certaines villes.. À titre d'exemple, le profil d'une commune type présentant une dominance de Papier (24,02 %) et de Plastique (12,73 %) avec un faible taux d'Organique permet d'identifier immédiatement une zone urbaine dynamique où la consommation de produits emballés est élevée.

En conclusion, avec un MAE global de 4,3 % et un intervalle de confiance à 95 % très serré (compris entre 4,19 % et 4,41 %), le modèle démontre une précision remarquable. Cette marge d'erreur de seulement 0,22 point nous permet d'affirmer que la régression logistique multinomiale est capable de prédire la composition des déchets d'une nouvelle commune avec une fiabilité stable sur l'ensemble du territoire italien.

Concernant la régression logistique multinomiale intégrée au site, nous avons constaté une perte importante de fiabilité lors du passage en production. Alors que le modèle présentait des résultats cohérents en environnement local, son déploiement via l'API sur Render a révélé des instabilités techniques importantes. Par exemple, on a pu observé un exemple où il y avait une concentration anormale des prédictions sur quelques catégories (82,15% pour l'organique, 12,98% pour le papier), tandis que la majorité des autres classes (Plastique, Métal, Textile, etc.) restent bloquées à 0%.

6.1.d Comparaison des modèles

La comparaison s'effectue ici sur les modèles qui partagent les mêmes variables de prédiction. Nous comparerons, dans un premier temps,la régression linéaire multiple avec le Random Forest effectué sans la variable région. Enfin, nous effectuerons aussi la comparaison entre les deux modèles intégrant la variable géographique.

Nous rappelons que pour l’ensemble des modèles, les données du jeu de données ont été mélangées pour que les modèles s'entraînent bien de manière aléatoire mais une “graine” a été implémentée pour que ce mélange de données soit le même. Nous avons opté pour le split 80% d'entraînement et 20% de tests, choisi pour maximiser la phase d'apprentissage de l'expert tout en conservant un échantillon de test statistiquement significatif pour garantir la fiabilité des statistiques de tests.

Comparaison entre la Régression linéaire multiple et le Random Forest

Pour la comparaison de la régression linéaire et du Random Forest , sans variable “region” donc, nous prenons en compte le code avec imputation par moyenne qui était plus performant sur les deux modèles.

Le premier constat est la supériorité nette du Random Forest (RF) sur la Régression Linéaire Multiple. Pour la catégorie "organic", le RF explique 47 % de la variance contre seulement 26 % pour la RLM. Pour le "glass", le score passe de 0.09 (RLM) à 0.23 (RF).

La RLM cherche une "ligne droite" (un hyperplan) pour expliquer les données. Or, les habitudes de tri et de consommation ne sont pas proportionnelles ; elles suivent des comportements de groupes. Le RF, en segmentant les données en "profils" de communes, capte cette réalité là où la RLM simplifie trop.

Le choix d'imputer les valeurs manquantes par la moyenne a un impact direct sur les modèles :

En RLM : L'imputation crée des points artificiels qui "aplatissent" la pente de la régression, ce qui fait chuter le R2. Le modèle peine à distinguer le vrai signal du bruit généré par la moyenne.

Avec Random Forest : Ce modèle est beaucoup plus robuste. Il est capable d'isoler les valeurs moyennes dans des branches spécifiques de ses arbres de décision sans perturber la prédiction des autres valeurs réelles.

Le Random Forest offre aussi une lecture beaucoup plus directe des enjeux.

L'analyse montre que :

La Population (pop) est le moteur principal de la répartition pour certaines classes (Organique et Verre).

Le Salaire (wage) devient prédominant pour d'autres classes (Papier, Bois, Métal).

Alors que la RLM donne des coefficients parfois difficiles à comparer entre eux, le RF hiérarchise l'importance en pourcentages clairs, facilitant l'aide à la décision.

Il est intéressant de noter que les deux modèles échouent sur les catégories RAEE (électronique) et Textile (R2 proche de 0).

Cela prouve que le taux de ces déchets ne dépend pas de la richesse ou du nombre d'habitants, mais probablement de facteurs externes non présents dans les données (proximité d'un point de collecte spécifique, politiques locales de tri, etc.).

Alors que la Régression Linéaire Multiple reste limitée par sa rigidité, le Random Forest permet de modéliser avec succès la complexité des comportements de tri.

Le modèle Random Forest est alors le modèle le plus intéressant dans le cas où nous ne disposons pas des régions.

Comparaison entre la Régression logistique multinomiale et le Random Forest

On rappelle que le coefficient de détermination R2 n’existe pas avec la régression logistique multinomiale. Nous basons alors nos comparaisons sur les scores MAE et NRMSE.

Pour le Random Forest, nous voyons que le MAE a régressé. On voit effectivement que la Random Forest a un MAE plus bas, c'est parce qu'il est capable de capturer des relations non-linéaires. Par exemple, l'effet du PIB sur les déchets n'est peut-être pas une ligne droite, mais une courbe qui stagne après un certain seuil.

On note aussi que le NRMSE est plus bas sur les petites classes (RAEE, Textile). Le Random Forest a tendance à "mieux réussir" sur les classes rares car il crée des sous-groupes spécifiques à ces cas, là où la régression essaie de faire passer une règle générale pour tout le monde.

Le Random Forest réduit significativement l'erreur moyenne absolue (MAE) pour l'ensemble des catégories. Cette amélioration est particulièrement visible sur les flux de déchets principaux :Déchets Organiques : La MAE chute de 8,94 avec la logistique à 6,69 avec le RF.

Métaux : L'erreur est divisée par plus de deux, passant de 1,69 à seulement 0,73.

Le modèle RF capture mieux la réalité des données et commet des erreurs de prédiction beaucoup plus faibles en moyenne.

La différence la plus flagrante se situe au niveau de l'erreur relative normalisée (NRMSE), qui mesure la "dangerosité" de l'erreur par rapport à la moyenne :

Avec la régression logistique, l'erreur dépassait les 120 % à 150 % pour les électroniques et les textiles, rendant le modèle mathématiquement inutilisable (l'erreur moyenne était plus grande que la valeur prédite).

Le Random Forest, bien que confronté à la difficulté de ces données, ramène l'erreur entre 69 % et 86 %. Bien que perfectible, le modèle commence ici à identifier des structures logiques là où la logistique échoue totalement.

Le Random Forest (RF) est plus performant que la Régression Logistique sur la précision absolue dans toutes les classes de déchets

La régression logistique, par sa nature linéaire, ne parvient pas à saisir la complexité des interactions entre les variables (population, PIB, régions). À l'inverse, la structure non linéaire du Random Forest permet de diviser l'erreur relative par deux sur les classes les plus compliqué à prédire, s'imposant comme le modèle le plus robuste pour cette étude

En conclusion, le passage d'un modèle linéaire à l'approche non-linéaire du Random Forest a radicalement permis d'augmenter dans le bon sens nos résultats, comme en témoigne la chute brutale de l'erreur moyenne (MAE) sur l'ensemble des catégories. Cette progression est particulièrement flagrante sur les flux majeurs : pour les déchets organiques, on gagne plus de deux points de précision, tandis que pour le papier et le verre, l'erreur est carrément divisée par deux. Cette performance s'explique par la capacité de l'algorithme à saisir des interactions complexes comme les effets de seuil liés au PIB ou à la densité là où les modèles classiques s'obstinent à tracer des lignes droites. Plus impressionnant encore, le Random Forest parvient à stabiliser les "petits flux" (textile, électronique) qui étaient jusqu'ici impossibles à prédire. Là où la régression affichait des erreurs aberrantes dépassant les 120 %, le Random Forest ramène ces scores à des niveaux bien plus cohérents, s'imposant ainsi comme le modèle le plus robuste pour traduire la réalité du terrain.

6.2 Base de données 2 : CNN

6.2.a Résultats

Nous avons donc analyser nos modèles sur les images de test.

Modèle	Accuracy (%)	Loss	Temps (s)
Décroissant	53.12	3.21	314.55
Croissant	58.33	2.00	78.93
AlexNet	54.95	1.33	98.04
VGG	62.76	0.99	710.05

Figure 16: Résultats et Comparaison des modèles

Les performances montrent que le modèle VGG est le plus performant en termes de précision 62,76 %.Le loss est de 0,99 il mesure l'écart entre les prédictions du modèle et les vraies valeurs. Celui ci indiquant une meilleure capacité d'apprentissage et de généralisation. Cependant, il présente un temps d'exécution très élevé 710,05 secondes , ce qui le rend coûteux en calcul.

Le modèle AlexNet offre un bon compromis avec une loss faible 1,33 et un temps modéré 98,04 secondes, mais une précision légèrement inférieure 54,95 %.

Les modèles croissant et décroissant sont moins performants globalement. Le modèle croissant reste toutefois plus efficace que le décroissant avec une meilleure précision 58,33 % contre 53,12 %, une loss plus faible et un temps d'exécution bien inférieur

Les matrices de confusion nous ont permis de comprendre ou le modèle ce trompais.

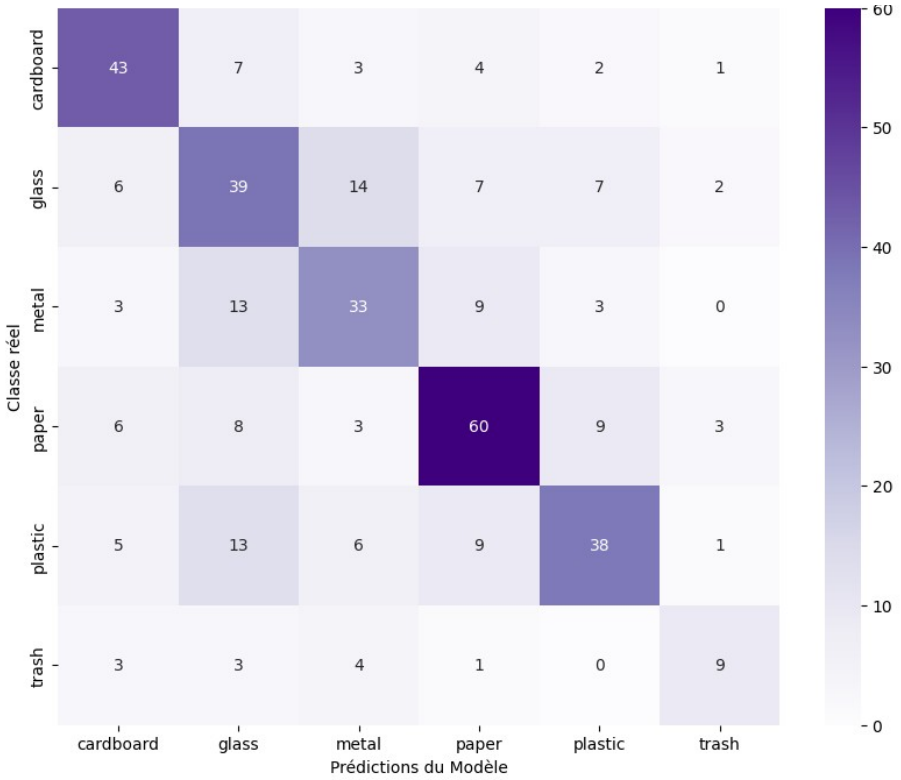

Figure 17: Matrice de Confusion - Modèle Décroissant

C'est le modèle le moin performant. La matrice montre une dispersion des erreurs sur presque toutes les catégories. Il a beaucoup de mal à distinguer le métal et le plastique du verre.Les prédictions sont moins franches, ce qui confirme les scores de Loss élevés vus précédemment.

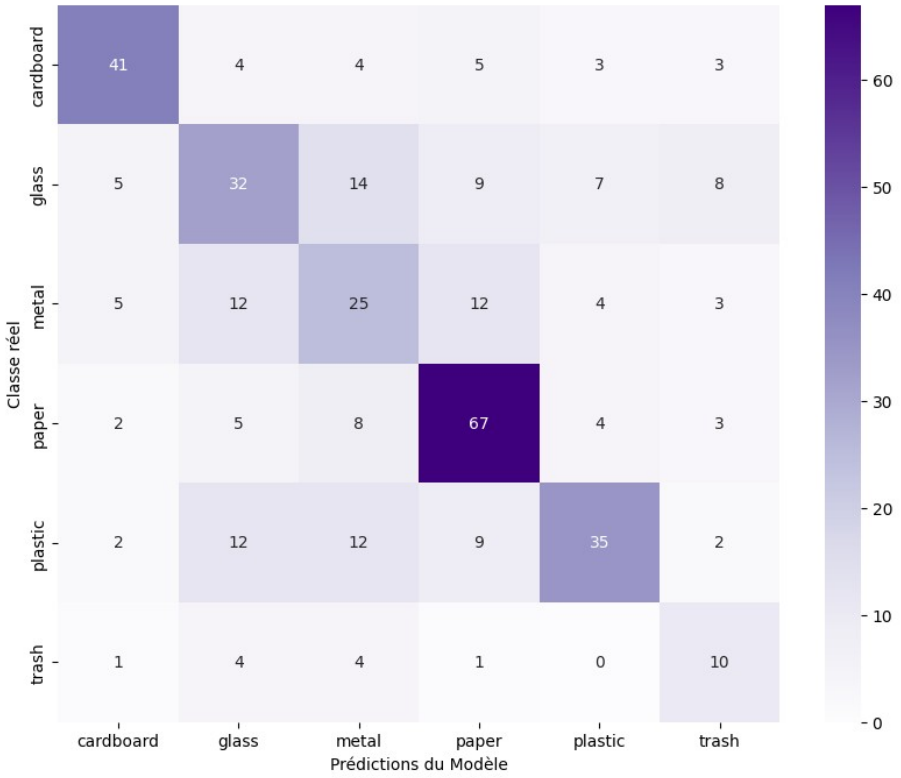

Figure 18: Matrice de Confusion - Modèle Croissant

Bien que moins profond que les architectures standards.On remarque qu'il est très efficace pour le papier. Il peine davantage sur le métal, qu'il confond régulièrement avec le verre.

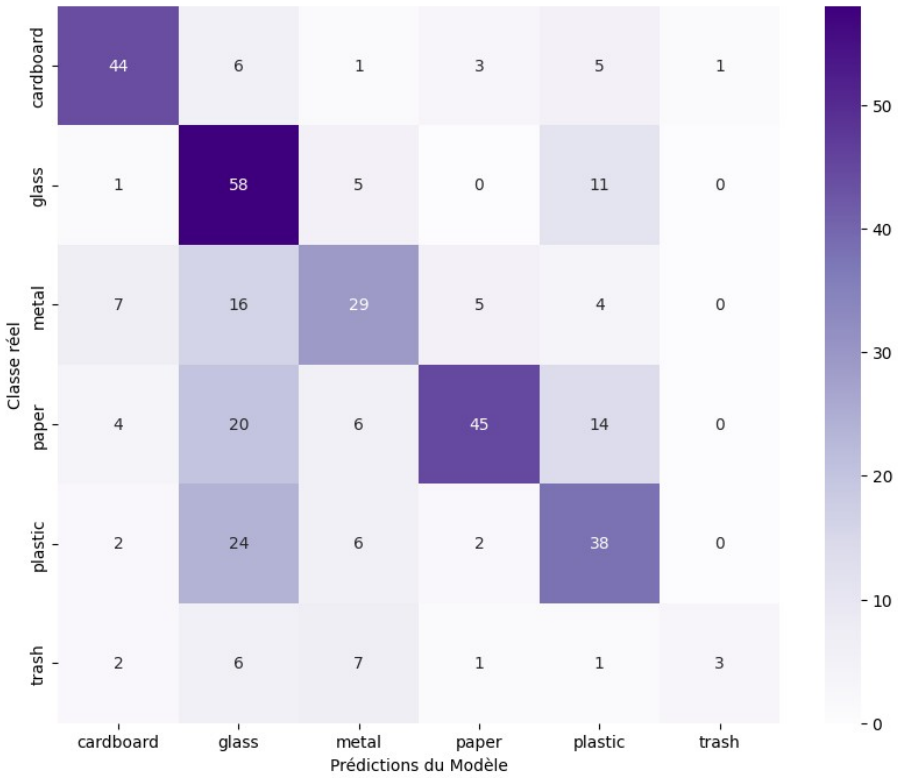

Figure 19: Matrice de Confusion - Modèle AlexNet

AlexNet obtient de bonnes performances globales, mais présente un biais. On constate une confusion importante, avec de nombreux échantillons classés à tort comme du verre, ce qui entraîne un taux élevé de faux positifs pour cette catégorie.

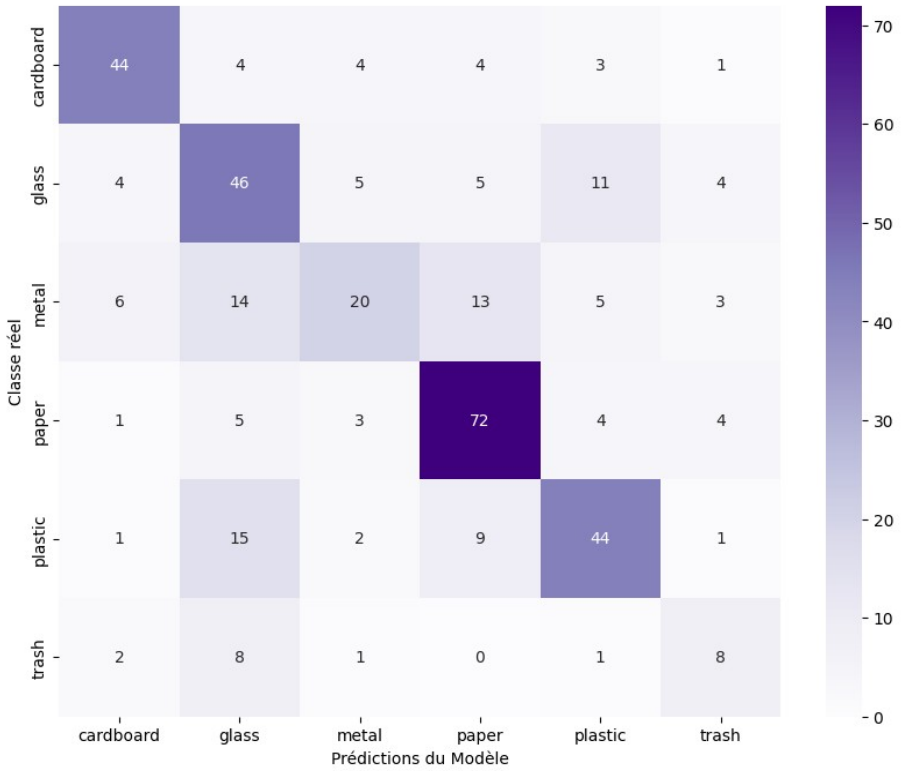

Figure 20: Matrice de Confusion - Modèle VGGNet

VGGNet s'impose comme le modèle le plus performant. Il présente la diagonale la plus marquée, ce qui signifie qu'il identifie correctement la majorité des classes. Il presente une bonne reconnaissance du papier 72 prédictions correctes sur 89. C'est le modèle qui disperse le moins ses erreurs, montrant une meilleure compréhension des caractéristiques visuelles des matériaux.

Les résultats initiaux n'étant pas à la hauteur de nos attentes, nous avons ré-entraîné nos modèles les plus performants sur un jeu de données plus vaste. Cette approche nous a permis de conserver une méthodologie constante tout en offrant au modèle une base d'apprentissage plus représentative.

Comparaison des modèles

Modèle	Accuracy (%)	Loss	Temps (s)
Croissant	88.05	0.8454	2547.83
VGG	79.11	0.656	4582.15

Figure 21: Résultats sur la seconde base de données

Nous avons réussi à améliorer la précision de nos modèles. En effet l'analyse des résultats montre que le modèle Croissant est nettement plus performant que l'architecture VGG sur ce jeu de données. Avec une précision de 88,05 %, il surpasse VGGNet de près de 9 points. De plus, il s'avère bien plus efficace en termes de ressources, avec un temps d'exécution de 2547 s, soit environ 1,8 fois plus rapide que VGGNet.

Modèle	Accuracy (%)
VGG + Croissant	90.54

Figure 22: Résultats VGGnet + Croissant

L'étape finale de notre démarche a consisté à fusionner nos deux architectures via un système de vote. Cette combinaison VGGNet + Croissant a permis de compenser les erreurs spécifiques de chaque modèle, élevant l'accuracy globale à 90,54 %.

6.2.b Discussion & Conclusion

Dans ce projet, nous avons appliqué plusieurs architectures de CNN pour la classification de nos données. Les résultats obtenus ont été décevants. Malgré nos efforts en parallèle pour les améliorer, avons tenté diverses stratégies :

- l’augmentation des données afin d’élargir le jeu d’entraînement.
- un rééquilibrage des classes pour limiter le biais.
- l’augmentation du nombre de couches et du nombre d’époques pour permettre un apprentissage plus profond.

Cependant, ces ajustements n'ont pas conduit à des améliorations significatives, tout en entraînant un temps de calcul très élevé.

Cette limitation peut s'expliquer par plusieurs facteurs.

Qualité et complexité des données, certaines classes sont naturellement difficiles à distinguer, ce qui limite les performances des modèles. Egalement la taille de notre Dataset rentre en compte. Même avec de la data augmentation, le réseau peut rester sous-entraîné ou sur-apprendre certaines classes.

Limites de l'architecture choisie, certains CNN, comme AlexNet, ont un nombre de paramètres fixe qui peut ne pas suffire pour capturer toute la variabilité des données modernes. Augmenter les couches et les époques améliore rarement les performances si le dataset ou la du structure du réseau ne permet pas un apprentissage efficace.

En résumé, les performances limitées semblent liées à une combinaison de la pauvreté de nos données et des contraintes inhérentes à l'architecture et aux ressources disponibles.

De plus une compréhension théorique plus approfondie nous permettrais de mieux exploiter comprendre Le Deep Learning, et plus particulièrement les CNN. Ces concepts mathématiques et structurels précis que nous avons hate de développer dans la suite de notre cursus.

Des approches alternatives, telles que l'utilisation de modèles plus récents ResNet, EfficientNet ,pourraient également offrir de meilleurs résultats.

Chapitre 7

7 Site Web

L'aboutissement de ce projet est une plateforme interactive intitulée Tri Facile. Ce site web ne se contente pas d'afficher des données : il sert d'interface entre nos modèles d'intelligence artificielle complexes et l'utilisateur (citoyen ou décideur municipal).

7.1.a Architecture et Rendu

Le déploiement de notre solution a représenté un défi technique majeur, car il a fallu faire cohabiter trois environnements différents : un serveur web pour l'interface, une API Python pour l'IA, et une API R pour les prédictions statistiques.

Pour le backend, nous avons fait le choix de la plateforme Render avec l'utilisation de deux API distinctes.

L'API Python gère le modèle CNN pour la reconnaissance d'images, déployée de manière native pour garantir une exécution rapide des bibliothèques TensorFlow et Keras.

Concernant l'API R (Plumber via Docker), les modèles de Random Forest, de régression linéaire et de régression multinomiale ayant été développés sous R, nous avons utilisé Docker pour son déploiement. Nous avons créé un Dockerfile spécifique pour encapsuler l'environnement R et ses dépendances.

Initialement, nous avions envisagé de traiter la régression linéaire multiple sous Python, cependant nous avons préféré centraliser tous les modèles statistiques sous R.

Ce choix nous a permis d'affiner notre régression linéaire multiple en y intégrant des variables qualitatives, comme les régions. Grâce à la fonction lm de R, ces variables catégorielles sont automatiquement traitées comme des variables binaires (dummy variables), attribuant une valeur de 1 ou 0 selon l'appartenance à une région. Cette méthode permet au modèle de mélanger efficacement des données numériques, telles que la population ou les revenus, avec des données géographiques, offrant ainsi une analyse beaucoup plus précise que si nous nous étions limités à des variables uniquement quantitatives.

Pour la partie visible du projet, nous avons utilisé InfinityFree pour héberger notre site en ligne et nous y avons rattaché notre base de données MySQL. L'une des étapes clés a été de configurer le site pour qu'il puisse interroger cette base tout en restant synchronisé avec les données traitées par nos API distantes. Pour compenser la mise en veille automatique de nos serveurs sur Render, nous avons mis en place un système de “réveil” : dès le chargement de la page d'accueil, une requête silencieuse est envoyée aux API pour les réactiver. Cela permet de réduire considérablement le temps d'attente lors de la première utilisation, évitant un délai de chargement de 30 secondes pour les prédictions.

Enfin, nous avons connecté le site sur InfinityFree aux deux API distantes, ce qui permet à l'utilisateur d'obtenir des réponses instantanées de nos moteurs de prédiction directement sur l'interface.

7.2 Fonctionnalités

L'interface de Tri Facile a été conçue pour offrir une expérience utilisateur complète, allant de la simple consultation d'informations à l'interaction directe avec nos modèles d'intelligence artificielle.

La première fonctionnalité est la carte interactive de l'Italie, qui permet de visualiser géographiquement les données de notre base de données. En naviguant sur cette carte, l'utilisateur peut identifier rapidement les disparités territoriales. Un menu déroulant permet de filtrer la visualisation selon différents critères, comme la richesse (revenu moyen), le taux de tri, la densité de population, ou encore la quantité de déchets, afin de mettre en évidence les corrélations visuelles avant même d'utiliser les modèles de prédiction.

En cliquant sur une région, comme Piémont, un panneau détaillé s'affiche. Il regroupe les indicateurs clés que nous avons exploités dans nos modèles : le taux de tri par rapport à la moyenne nationale, la population, le revenu moyen par habitant, ainsi que des données géographiques comme l'altitude. Cette interface permet donc de rendre nos données brutes accessibles et compréhensibles pour n'importe quel utilisateur.

Figure 23: extrait de la carte des données - site Tri Facile

Pour approfondir cette analyse, nous avons intégré un onglet « Graphiques » pour visualiser nos graphiques fait durant la partie « Analyse exploratoire des données » ainsi que leurs analyses mais aussi une option permettant à l'utilisateur de personnaliser ses propres graphiques . En effet, l'interface permet de sélectionner un certain nombre de région, à la guise de l'utilisateur, ainsi que différentes variables tels que la densité, le taux de déchets triés, l'altitude, le coût total par habitant etc… en fonction de toutes les régions. Les graphiques peuvent s'afficher en barres comme sur l'exemple ci-dessous ou alors en forme en « radar ». Enfin la fonction nuage de points permet de croiser deux de ses variables et ainsi d'observer des corrélations entre deux de ces variables

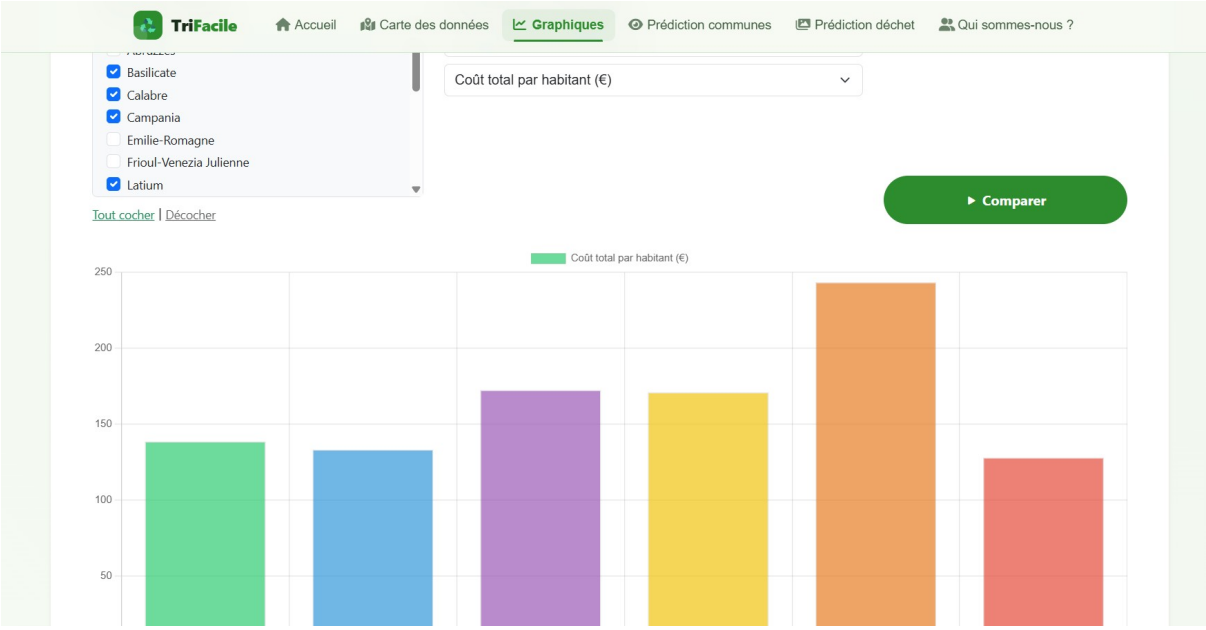

Figure 24: extrait des graphiques interactifs - site Tri Facile

Ensuite, nous avons mis en place un simulateur de production de déchets. En s'appuyant sur nos modèles de régression et de Random Forest entraînés sous R, cette fonctionnalité permet d'estimer la composition des déchets d'une commune. Nous avons également ajouté une prédiction de la quantité totale de déchets pour une commune.

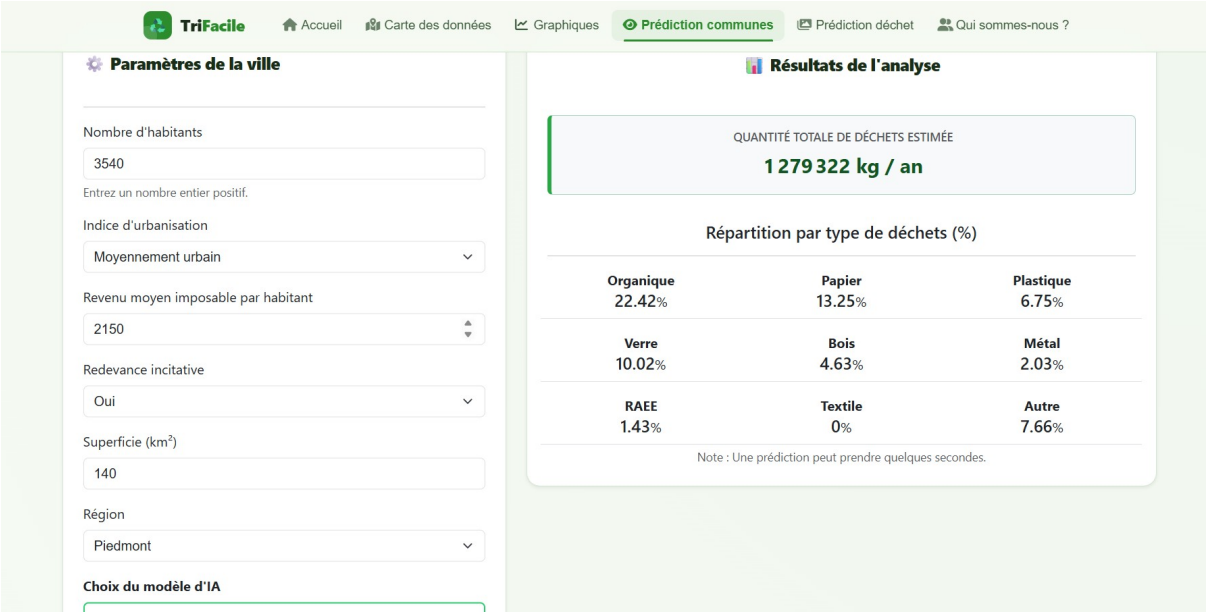

Figure 25: extrait de la prédiction des communes - site Tri Facile

Enfin, nous avons développé un module d'aide au tri basé sur notre modèle de vision par ordinateur (CNN). Cette fonctionnalité permet à l'utilisateur de soumettre une image de son déchet pour obtenir une classification instantanée du matériau (Plastique, Verre, Papier,

etc.). Afin d'améliorer continuellement la précision de notre IA, nous avons également intégré un module de contestation. Si l'utilisateur constate que le modèle s'est trompé, il peut sélectionner manuellement la bonne catégorie de déchet. Cette information est alors directement renvoyée dans notre base de données. Sur le long terme, nous envisageons d'utiliser ces données corrigées pour ré-entraîner périodiquement le modèle, créant ainsi un système auto-apprenant qui gagne en fiabilité au fur et à mesure de son utilisation.

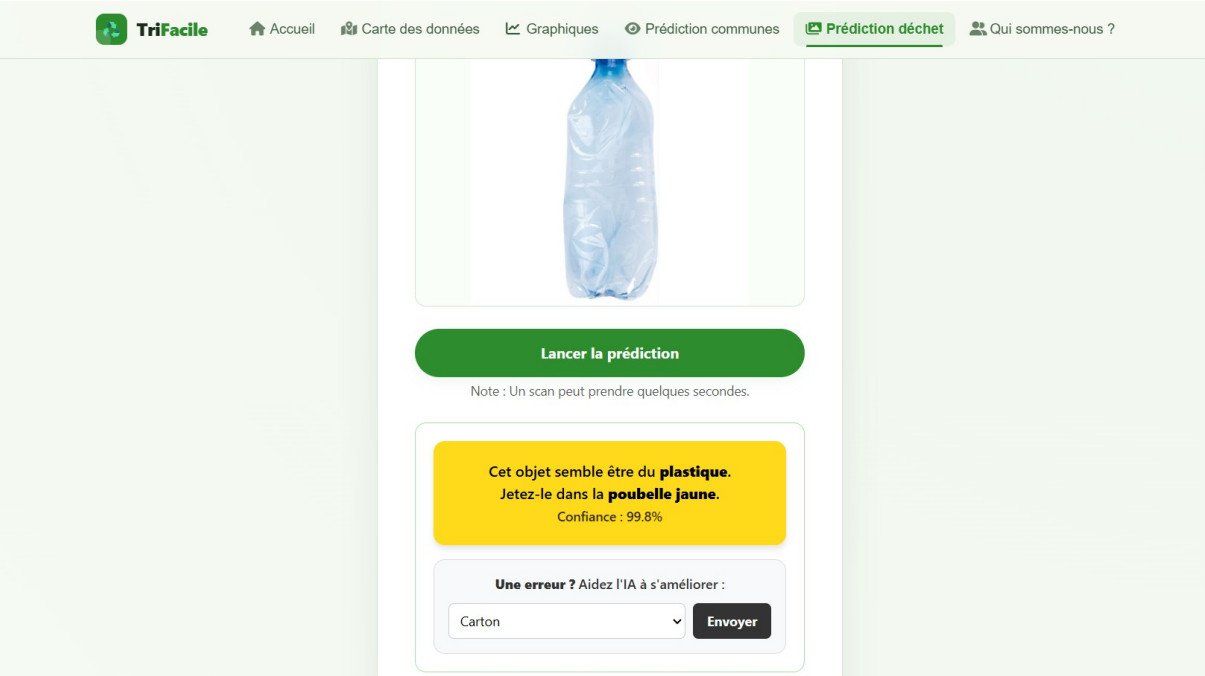

Figure 26: extrait de la prédiction de déchets - site Tri Facile

Chapitre 8

8 Conclusion et Perspectives

8.1 Conclusion

Le projet Tri Facile a permis de démontrer que l'intelligence artificielle constitue un levier stratégique pour accompagner la transition écologique en transformant des données complexes en outils d'aide à la décision. À travers l'analyse de données provenant de plus de 4 300 communes italiennes, nos travaux ont mis en évidence que la performance du tri ne dépend pas uniquement de la volonté citoyenne, mais résulte d'une combinaison de facteurs géographiques, socio-économiques et législatifs. L'utilisation de modèles de Random Forest a notamment prouvé l'efficacité des politiques incitatives, révélant que les municipalités appliquant une redevance au poids (PAYT) obtiennent des taux de recyclage systématiquement plus élevés que celles utilisant une taxe fixe.

Nos recherches ont également souligné l'importance de la morphologie urbaine et de la géographie, avec une fracture nette entre les régions du Nord et du Sud de l'Italie, ainsi qu'une difficulté accrue pour le tri dans les zones à forte densité de population ou de haute altitude. En parallèle, le développement d'un réseau de neurones convolutif (CNN) pour la reconnaissance d'images a offert une solution concrète pour lever les doutes des habitants lors du geste de tri. L'aboutissement de ce travail est une plateforme interactive qui lie l'analyse statistique et la vision par ordinateur pour offrir un service complet aux municipalités et aux citoyens.

8.2 Perspectives

Malgré des résultats probants, plusieurs axes de développement permettraient d'enrichir les capacités de Tri Facile à l'avenir. Une première évolution consisterait à élargir le périmètre géographique au-delà de l'Italie, en intégrant notamment des données françaises pour

adapter nos modèles aux spécificités réglementaires et logistiques locales. Il serait également pertinent d'ajouter de nouvelles variables explicatives au simulateur, comme la proximité des centres de tri ou l'impact des campagnes de sensibilisation. En effet, bien que notre base de données actuelle inclut déjà des flux comme le textile ou les équipements électroniques (RAEE), ces derniers restent aujourd'hui difficiles à prédire avec précision car ils dépendent de facteurs locaux et comportementaux que les seules variables macro-économiques ne suffisent pas à capturer.

Sur le plan technique, nous devons stabiliser nos modèles en ligne. La régression logistique, bien qu'efficace en test, nécessite d'être ajustée pour corriger les erreurs de prédiction apparues lors du déploiement. L'objectif est de recalibrer le modèle pour qu'il redevienne capable de distinguer précisément chaque catégorie de déchet, sans en oublier. Parallèlement, la base de données dédiée à la reconnaissance d'images gagnerait à être affinée avec des catégories plus précises, permettant de distinguer des objets spécifiques comme un pot de yaourt ou un sachet de thé pour fournir des consignes de tri encore plus exactes. Enfin, le module de contestation déjà intégré au site web ouvre la voie vers un système auto-apprenant où les corrections apportées par les utilisateurs serviraient à ré-entraîner périodiquement les modèles. À terme, Tri Facile pourrait devenir un outil de gestion opérationnelle pour une commune, lui permettant de prédire ses volumes de déchets pour optimiser ses collectes tout en engageant ses habitants de manière ludique et efficace dans la préservation de l'environnement.

Références

[1] Di Foggia, Giacomo. Municipal waste management cost and fee schemes, 2022, https://doi.org/10.17632/w5f9kg7743.1.

[2] Cofone, Leonardo. Waste Images Dataset (2000/class), 2025, .

[3] Yann LeCun, Léon Bottou, Yoshua Bengio et Patrick Haffner, Gradient-based learning applied to document recognition, 1998

[4] David H. Hubel et Torsten N. Wiesel, Receptive fields and functional architecture of monkey striate cortex, 1968

[5] Krizhevsky, A., Sutskever, I., & Hinton, G. E., ImageNet classification with deep convolutional neural networks., 2012.

[6] Simonyan, Karen; Zisserman, Andrew, Very Deep Convolutional Networks for Large-Scale Image Recognition, 2014

Bibliographie

Ouvrages et Thèses :

- Haton, J.-P., Haton, E., & Haton, M.-C. (2023). Intelligences artificielles : de la théorie à la pratique - Modèles, applications et enjeux des IA. Dunod.
- Pasquet, J. (2016). Modélisation, détection et classification d'objets urbains à partir d'images photographiques aériennes [Thèse de doctorat, Université de Montpellier].

Cours et Supports Pédagogiques :

- Cours TE61MI - Modélisation et Méthodes Statistiques - S.Lèbre (2026)
- Cours TE54MI - Science des données 3 - B.Bourrel, M.Servajean (2025)
- Cours TE5PR1MI - Gestion de projet - N.Patel, T.Michel-Picque (2025)
- Cours TE31MI - Science des données 2 - M.Demangeot (2025)
- Cours TE43MI - Base de données - S.Bringay (2025)
- Cours TE34MI - Science des données 1 - N.Khiari (2024)

Ressources en ligne :

- Wikipédia. (2024). Régression logistique. Consulté sur https://fr.wikipedia.org/wiki/Régression_logistique

Outils d'Intelligence Artificielle :

- Google. (2024). Gemini (Version 1.5 Pro) [Large Language Model]. https://gemini.google.com/

Annexe

Nom colonne	Type	Signification	Caractéristique
region	Chaîne de caractère	Région	Variable qualitative (ex: Lombardie, Sicile)
province	Chaîne de caractère	Province	Subdivision de la région
name	Chaîne de caractère	Nom de la commune	
tc	Double	Coût total de gestion des déchets	En € par habitant
cres	Double	Coût spécifique des déchets résiduels	Lié aux déchets non triés par habitant
csor	Double	Coût spécifique des déchets triés	Lié à la gestion de la collecte sélective
istat	Nombre	Code ISTAT	Code national d'identification officiel
area	Float	Superficie de la commune	Exprimée en km²
pop	Nombre	Population totale	Nombre d'habitants de la commune
alt	Nombre	Altitude	En mètres au-dessus du niveau de la mer
isle	Binaire (tinyint)	Indicateur d'île	1 = commune insulaire ; 0 = sinon
sea	Binaire	Localisation littorale	1 = commune en bord

	(tinyint)		de mer ; 0 = sinon
pden	Float	Densité de population	Nombre d'habitants par km²
wden	Double	Densité de déchets	Quantité totale de déchets produite par km²
urb	Nombre	Indice d'urbanisation	1 = faible ; 2 = moyen ; 3 = fort
fee	Chaîne de caractère	Schéma de tarification	Type de redevance appliquée
d_fee	Binaire	Redevance incitative (PAYT)	1 = appliquée , 0 = forfait classique
sample	Binaire	Échantillon spécifique	1 = zone utilisant la redevance incitative ; 0 = sinon
organic	Float	Part organique	% de déchets organiques collectés
paper	Float	Part papier/carton	% de papier et carton collectés
glass	Float	Part verre	% de verre collecté
wood	Float	Part bois	% de déchets en bois collectés
metal	Float	Part métaux	% de déchets métalliques collectés
plastic	Float	Part plastique	% de déchets plastiques collectés
raee	Float	Part DEEE	% de déchets d'éq. électriques et

			électroniques
texile	Float	Part textile	% de déchets textiles collectés
other	Float	Part autres	% de déchets non classifiés collectés
msw_so	Nombre	Déchets triés totaux	Quantité totale collectée séparément (en kg)
msw_un	Nombre	Déchets non triés totaux	Quantité de déchets résiduels (en kg)
msw	Nombre	Totale déchets municipaux	Somme totale des déchets produits (en kg)
sor	Double	Taux de tri sélectif	Ratio (part) des déchets triés sur le total
geo	Nombre	Zone géographie	1 = Sud ; 2= Centre ; 3 = Nord
roads	Nombre	Réseau routier	Longueur totale des routes (en km)
s_wteregio	Float	Valorisation énergétique	% envoyé vers des usines d'incinération
s_landfill	Float	Mise en décharge	% de déchets envoyés en décharge
gdp	Float	Revenus municipaux	Niveau de revenus par habitant (en €)
proads	Float	Rapport habitants/routes	Densité de population par km de route (en log)
wage	Float	Revenu imposable	Moyenne des revenus imposables (en €)

finance	Float	Revenus globaux	Capacité financière de la ville (en €)

```python
seed = 25
epoch = 20
num_classes = 6

os.environ['PYTHONHASHSEED'] = str(seed)
random.seed(seed)
np.random.seed(seed)
tf.keras.utils.set_random_seed(seed)

tf.config.experimental.enable_op_determinism()

base_path = "Projet_SDD_4/data/data_split"
img_height= 224
img_width = 224
batch_size = 32

train_data = tf.keras.utils.image_dataset_from_directory(
    base_path +'/train',
    image_size=(img_height, img_width),
    batch_size=batch_size,
    shuffle=True,
    seed = seed,
)

val_data = tf.keras.utils.image_dataset_from_directory(
   base_path +'/val',
    seed=seed,
    image_size=(img_height, img_width),
    batch_size=batch_size
)

test_data = tf.keras.utils.image_dataset_from_directory(
    base_path +'/test',
    image_size=(img_height, img_width),
    batch_size=batch_size,
    shuffle=False
)
class_names = val_data.class_names
```

Figure 27: Code python - Protocole expérimental

```
start_time_dcr = time.time()

model_dcr = tf.keras.Sequential([
    layers.Rescaling(1./255),
    layers.Conv2D(128,4, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64,4, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32,4, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(16,4, activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(64,activation='relu'),
    layers.Dense(num_classes, activation='softmax')
])

model_dcr.compile(optimizer='adam',
    loss=tf.losses.SparseCategoricalCrossentropy(from_logits=False),
  metrics=['accuracy'],)

callback_modele1 = tf.keras.callbacks.TensorBoard(log_dir="logs/model_dcr")

history_dcr = model_dcr.fit(
    train_data,
    validation_data=val_data,
    epochs=epoch,
    callbacks=[callback_modele1],
    verbose=0
)

end_time_dcr = time.time()
```

Figure 28: Code - Modèle Décroissant

```
start_time_cr = time.time()

model_cr = tf.keras.Sequential([
    layers.Rescaling(1./255),
    layers.Conv2D(16,4, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32,4, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64,4, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(128,4, activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(64,activation='relu'),
    layers.Dense(num_classes, activation='softmax')
])

model_cr.compile(optimizer='adam',
                 loss=tf.losses.SparseCategoricalCrossentropy(from_logits=False),
                   metrics=['accuracy'],)

tensorboard_callback = tf.keras.callbacks.TensorBoard(log_dir="logs/model_cr")

history_cr = model_cr.fit(
    train_data,
    validation_data=val_data,
    epochs=epoch,
    callbacks=[tensorboard_callback],
    verbose=0
    )

end_time_cr = time.time()
```

Figure 29: Code - Modèle Croissant

```python
start_time_alex = time.time()

model_alex =  tf.keras.Sequential([
    layers.Rescaling(1./255, input_shape=(img_height, img_width, 3)),
    layers.Conv2D(filters=96, kernel_size=(11, 11), strides=(4, 4), activation='relu', padding='valid'),
    layers.MaxPooling2D(pool_size=(3, 3), strides=(2, 2)),
    layers.BatchNormalization(),
    layers.Conv2D(filters=256, kernel_size=(5, 5), strides=(1, 1), activation='relu', padding='same'),
    layers.MaxPooling2D(pool_size=(3, 3), strides=(2, 2)),
    layers.BatchNormalization(),
    layers.Conv2D(filters=384, kernel_size=(3, 3), strides=(1, 1), activation='relu', padding='same'),
    layers.Conv2D(filters=384, kernel_size=(3, 3), strides=(1, 1), activation='relu', padding='same'),
    layers.Conv2D(filters=256, kernel_size=(3, 3), strides=(1, 1), activation='relu', padding='same'),
    layers.MaxPooling2D(pool_size=(3, 3), strides=(2, 2)),
    layers.BatchNormalization(),
    layers.Flatten(),
    layers.Dense(4096, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(4096, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])
model_alex.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = model_alex.fit(
    train_data,
    #augmented_train_data,
    validation_data=val_data,
    epochs=epoch,
    verbose=0

)
end_time_alex = time.time()
```

Figure 30: Code - Modèle AlexNet

```python
start_time_vgg = time.time()
model_vgg =  models.Sequential([
    layers.Input(shape=(img_height, img_width, 3)),
    layers.Rescaling(1./255),
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.MaxPooling2D((2, 2), strides=(2, 2)),
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.MaxPooling2D((2, 2), strides=(2, 2)),
    layers.Conv2D(256, (3, 3), padding='same', activation='relu'),
    layers.Conv2D(256, (3, 3), padding='same', activation='relu'),
    layers.Conv2D(256, (3, 3), padding='same', activation='relu'),
    layers.MaxPooling2D((2, 2), strides=(2, 2)),
    layers.Conv2D(512, (3, 3), padding='same', activation='relu'),
    layers.Conv2D(512, (3, 3), padding='same', activation='relu'),
    layers.Conv2D(512, (3, 3), padding='same', activation='relu'),
    layers.MaxPooling2D((2, 2), strides=(2, 2)),
    layers.Conv2D(512, (3, 3), padding='same', activation='relu'),
    layers.Conv2D(512, (3, 3), padding='same', activation='relu'),
    layers.Conv2D(512, (3, 3), padding='same', activation='relu'),
    layers.MaxPooling2D((2, 2), strides=(2, 2)),
    layers.Flatten(),
    layers.Dense(4096, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(4096, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])
model_vgg.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history_vgg = model_vgg.fit(
    train_data, validation_data=val_data, epochs= epoch,verbose=0
)
end_time_vgg = time.time()
```

Figure 31: Code - Modèle VGGNet